

Post-Quantum Secure and Dynamic Load-Balanced Encryption for IIoT Data in Fog Computing

Phinnawat Yaemsanguan, Rangsimann Sattayarom, Ratchanon Wongwitutai, Somchart Fugkeaw
Sirindhorn International Institute of Technology, Thammasat University, Thailand
6622770764@g.siit.tu.ac.th, 6622771747@g.siit.tu.ac.th, 6622780268@g.siit.tu.ac.th

Abstract—Industrial Internet of Things (IIoT) systems generate high-volume data streams that increase device-to-cloud latency and impose heavy pressure on network and cloud infrastructures. Fog computing addresses this challenge by moving computation and data processing closer to the network edge. However, conventional secure fog-assisted IIoT frameworks often focus mainly on encryption while overlooking the computational overhead introduced by post-quantum cryptography and the heterogeneous resource constraints of fog nodes. These challenges become more significant in practical deployments, where fog nodes differ in CPU capability, memory capacity, and network conditions, causing static load-sharing mechanisms to perform poorly under bursty traffic.

In this work, a post-quantum fog-assisted security architecture for IIoT is proposed. The system integrates PUF-based device authentication, Kyber-based post-quantum key establishment during the setup phase, and ChaCha20-Poly1305 for lightweight authenticated protection of IIoT data streams. Multiple protected data streams generated by IIoT devices are collected and aggregated at the edge layer, where a group-based authentication mechanism is applied to reduce repeated authentication overhead before forwarding the validated batch to the fog layer. To support fine-grained access control under resource-constrained fog environments, a split-phase lattice-based CP-ABE mechanism is adopted, in which sensitive cryptographic operations are executed within Trusted Execution Environments (TEEs) while complementary processing is handled outside the enclave. After secure aggregation, the fog layer re-encrypts the protected batch using AES for subsequent storage and controlled access. In addition, a dynamic load-balancing mechanism coordinated by a Master Fog Node assigns security-related tasks to heterogeneous fog nodes according to real-time resource availability. The proposed design aims to improve secure data handling efficiency while maintaining post-quantum protection and scalable fog-layer processing for IIoT environments.

Index Terms—Industrial Internet of Things (IIoT), Fog Computing, Post-Quantum Cryptography (PQC), Lattice-Based CP-ABE, Dynamic Load Balancing, Trusted Execution Environment (TEE), Physical Unclonable Function (PUF), Fine-Grained Access Control.

I. INTRODUCTION

Industrial Internet of Things (IIoT) environments consist of large numbers of sensors, machines, and controllers that continuously generate data at high volume and velocity. As industrial systems scale, the amount of streaming data increases significantly, placing heavy pressure on communication networks and centralized cloud infrastructures. Transmitting raw IIoT data directly to the cloud often leads to bandwidth congestion, increased latency, and inefficient use of network and computing resources. Fog computing addresses these

limitations by relocating computation and storage resources closer to IIoT devices, thereby reducing communication delay and network load.

However, ensuring secure and efficient data handling in fog-assisted IIoT environments remains a major challenge. Many existing approaches focus primarily on data encryption while giving limited attention to the computational overhead of cryptographic protection, especially in high-volume and real-time industrial settings. In practice, IIoT data streams must be authenticated, protected, aggregated, and forwarded across multiple layers under strict latency constraints. Treating all tasks uniformly can introduce unnecessary overhead and reduce system efficiency, particularly when heterogeneous processing resources are involved.

These challenges become more critical in the post-quantum era. Post-quantum cryptographic mechanisms generally incur higher computational and communication overhead than conventional public-key schemes, making their direct deployment difficult in large-scale IIoT systems. Moreover, improper placement of post-quantum operations across the IIoT–fog–cloud pipeline may increase end-to-end latency, reduce throughput, and overload resource-constrained components. Therefore, designing an efficient security workflow that determines how authentication, key establishment, encryption, and access control should be distributed across the edge, fog, and cloud layers becomes an important research problem.

The problem is further complicated by the heterogeneous nature of fog environments. Fog nodes often differ in processing capability, memory capacity, and network conditions, while Trusted Execution Environments (TEEs) provide limited secure memory for sensitive cryptographic operations. In practical deployments, each fog node can host multiple concurrent TEE instances (enclaves), enabling parallel execution of security-sensitive workloads. However, this multi-enclave capability introduces additional challenges in enclave memory management, workload isolation, and scheduling efficiency, particularly under dynamic and high-throughput IIoT conditions. As a result, improper workload placement may lead to enclave contention, EPC memory pressure, and performance degradation, making TEEs a potential throughput bottleneck despite their strong security guarantees. Consequently, efficient workload management must operate at multiple levels, including inter-node load balancing across heterogeneous fog nodes and intra-node scheduling between TEE and REE execution environments. This necessitates an intelligent, enclave-

aware scheduling mechanism capable of jointly optimizing performance, security, and resource utilization under dynamic system conditions. In many current deployments, workload distribution is still handled using static load balancing or fixed routing strategies, such as threshold-based offloading or pre-determined forwarding rules. These approaches cannot adapt effectively to bursty traffic, changing network conditions, or diverse fog-node capabilities. As a result, computationally intensive security tasks may be assigned to unsuitable fog nodes, creating bottlenecks, queue buildup, and service degradation.

To address these challenges, this work proposes a post-quantum fog-assisted security architecture for IIoT environments. The proposed system integrates PUF-based device authentication, Kyber-based post-quantum key establishment during the setup phase, and ChaCha20-Poly1305 for lightweight authenticated protection of IIoT data streams. Multiple protected streams are collected and aggregated at the edge layer, where a group-based authentication mechanism is applied to reduce repeated verification overhead before forwarding validated batches to the fog layer. To support secure storage and fine-grained access control, the fog layer re-encrypts aggregated protected data using AES and applies a split-phase lattice-based CP-ABE mechanism, in which sensitive cryptographic operations are executed within Trusted Execution Environments while complementary processing is handled outside the enclave. In addition, a dynamic load-balancing mechanism coordinated by a Master Fog Node assigns security-related workloads according to real-time resource availability. Through this design, the proposed system aims to improve secure data handling efficiency while maintaining low-latency processing and post-quantum resilience in heterogeneous fog-assisted IIoT environments.

In summary, the main contributions of this work are as follows:

1) **Post-Quantum and TEE-Assisted Secure IIoT Architecture**

We design an end-to-end fog-assisted IIoT security architecture integrating PUF-rooted authentication, CRYSTALS-Kyber key establishment, ChaCha20-Poly1305 authenticated encryption, and split-phase lattice-based CP-ABE. Security-sensitive cryptographic operations are confined within TEEs, while non-sensitive policy expansion is securely offloaded to the REE, improving secure processing efficiency under post-quantum protection.

2) **Secure and Scalable Micro-Batch Ingestion**

We introduce a gateway-assisted validation and deterministic micro-batch aggregation mechanism that performs lightweight packet authentication, malicious packet filtering, and integrity-protected batch formation before fog-layer processing. The proposed design reduces repeated cryptographic verification overhead and improves ingestion scalability for high-throughput IIoT streams.

3) **Enclave-Aware and Stability-Aware MFN Selection**

We propose a capability-aware Master Fog Node (MFN)

selection mechanism that jointly considers enclave-level queue dynamics, EPC pressure, runtime trustworthiness, resource availability, and temporal stability. Unlike conventional node-selection approaches relying only on coarse-grained metrics, the proposed mechanism enables more reliable coordination of security-sensitive workloads in heterogeneous fog environments.

4) **Spider: Hierarchical Multi-Enclave Scheduling**

We propose **Spider**, a hierarchical enclave-oriented scheduling framework that jointly performs (i) inter-node workload orchestration across fog nodes and (ii) intra-node scheduling across multiple parallel TEE enclaves. Spider integrates workload profiling, EPC-aware scheduling, trust-aware routing, computation-reuse optimization, and enclave-parallel batch decomposition to mitigate enclave contention and improve secure processing throughput.

5) **Decentralized Fault-Tolerant Secure Recovery**

We design a group-based failure detection and secure workload recovery mechanism that supports authenticated heartbeat monitoring, quorum-based failure confirmation, and lightweight enclave-state reassignment under fog-node failures. The proposed recovery mechanism minimizes recomputation overhead while preserving secure and continuous workload execution in dynamic fog-assisted IIoT environments.

II. RELATED WORK

A. Fog-Assisted IIoT Security and TEE-Based Execution

Fog computing has emerged as a key paradigm for enabling low-latency processing and distributed security services in Industrial Internet of Things (IIoT) systems. By bringing computation closer to end devices, fog nodes reduce communication overhead and improve responsiveness. However, ensuring secure authentication, fine-grained access control, and protected execution in distributed environments remains challenging.

Lightweight authentication in fog-assisted IIoT is commonly achieved using Physical Unclonable Function (PUF)-based mechanisms. For instance, [1], [2] propose PUF-based authentication schemes that support group authentication and efficient device verification with reduced computational overhead. While these approaches improve scalability and eliminate heavy key storage, they primarily focus on authentication and message integrity, without addressing fine-grained multi-user data sharing and access control.

To enable secure data sharing, Ciphertext-Policy Attribute-Based Encryption (CP-ABE) has been widely adopted. Works such as [3], [5] provide fine-grained access control and revocation in fog-assisted cloud environments, while [4] further introduces a lattice-based quantum-resistant CP-ABE scheme for IoT systems. Although these solutions strengthen confidentiality and access control, they operate independently of lightweight authentication mechanisms and do not consider hardware-assisted secure execution.

To protect sensitive computation, Trusted Execution Environments (TEEs) have been investigated at the edge and fog layers. Paper [6] identifies SGX-related limitations, including performance overhead, restricted enclave memory, and I/O security issues, while [7] shows that practical TEE implementations remain vulnerable to software, architectural, side-channel, and microarchitectural attacks. More broadly, fog-security frameworks [8]–[10] address trust-based secure computing, lightweight authentication and key management, and risk-aware access control, but none of them jointly integrate lightweight authentication, post-quantum access control, and TEE-assisted secure processing into a unified framework. Therefore, a gap remains for a unified fog-assisted IIoT framework that combines lightweight authentication, post-quantum fine-grained access control, and TEE-assisted secure processing.

B. Post-Quantum Cryptography for IIoT Systems

The rapid development of quantum computing threatens conventional public-key cryptographic schemes such as RSA and ECC, motivating the adoption of post-quantum cryptography (PQC). Among the major PQC families, lattice-based cryptography has gained significant attention because of its strong security foundation and computational efficiency.

CRYSTALS-Kyber and CRYSTALS-Dilithium have emerged as leading lattice-based candidates for key encapsulation and digital signatures, respectively. Prior studies [12], [13] demonstrate their practicality in IIoT and network-stack settings while also highlighting performance trade-offs. In particular, [12] integrates lattice-based ML-KEM and ML-DSA into TLS 1.3 and X.509 for IIoT systems, while [13] benchmarks Dilithium/Kyber and Falcon/Kyber stacks, showing trade-offs in execution cost, workload distribution, and bandwidth.

For resource-constrained environments, [14], [15] further examine the deployment of PQC in IoT and IIoT settings. In particular, [14] adapts CRYSTALS-Kyber to wireless and device-to-device communication for constrained devices, whereas [15] evaluates PQC algorithms in IIoT scenarios in terms of execution time and energy consumption, showing that most PQ KEMs are competitive with classical curves and that hybrid PQC-classical combinations introduce only limited additional overhead.

To strengthen hardware-level protection, [16] combines Kyber with PUF and TRNG mechanisms to reduce reliance on stored secrets and improve physical security. Complementary hardware-rooted works such as [17] also show that PUF-based authentication can strengthen resistance to impersonation and physical attacks in IoT settings, although they are not themselves full PQC deployment frameworks.

Broader studies such as [18], [19] discuss hybrid quantum-resistant architectures and PQC adoption in wider networked and edge-fog environments. These works emphasize issues such as crypto-agility, scalability, interoperability, and deployment complexity when integrating post-quantum protection into practical communication systems.

Despite these advances, existing studies mainly focus on standalone PQC deployment, protocol benchmarking, or authentication enhancement. The integration of Kyber/Dilithium-based protection into fog-assisted IIoT architectures, particularly in combination with hardware-bound authentication, fine-grained access control, and secure execution, still remains limited. This gap motivates the need for a more integrated fog-assisted PQC framework.

C. Load Balancing in Heterogeneous Fog Environments

Efficient load balancing is critical in fog-assisted IIoT systems because fog nodes are inherently heterogeneous in computational capability, latency, and energy profile. To improve system performance under such conditions, dynamic scheduling strategies have been widely adopted. Prior works such as [22] dynamically allocate workloads based on system conditions, while [24], [25] consider latency, energy consumption, and broader fog-cloud resource factors. Multi-objective optimization approaches such as [26] further attempt to balance performance metrics including delay, cost, and energy consumption.

More recent studies have explored the interaction between load balancing and security. For example, [27] incorporates energy-aware scheduling near industrial devices, while [28] introduces secure and energy-aware load balancing in an SDN-enabled fog environment. In addition, [29], [30] show that trusted execution environments such as TrustZone or TEE introduce non-negligible overhead. In particular, [29] shows that REE–TEE interaction and secure task execution increase runtime and energy cost, while [30] explicitly highlights cryptographic-operation overhead and the single-processor monopoly of TrustZone, both of which directly affect scheduling decisions.

Table I presents a feature-level comparison of existing approaches in fog-assisted IIoT environments. Prior works address important aspects such as fine-grained access control and revocation in [4], [5], and dynamic load balancing in fog systems in [22], [37], [39], [40]. In particular, the CP-ABE-based schemes in [4], [5] provide fine-grained access control and revocation capabilities, with [4] further supporting a quantum-resistant construction. However, these security-focused schemes do not consider the interaction between access control mechanisms and runtime resource scheduling in fog-assisted settings. Conversely, existing fog-based load balancing approaches [22], [37], [39], [40] demonstrate the effectiveness of dynamic task allocation in improving system performance, but they generally focus on latency, energy consumption, or workload distribution under conventional security assumptions. As a result, they do not explicitly account for the additional computational overhead introduced by post-quantum cryptography or hardware-assisted secure execution environments.

Overall, existing works provide valuable foundations in authentication, access control, and load balancing, but they still leave open challenges in understanding how security mechanisms, particularly lightweight authentication, post-quantum

TABLE I
COMPREHENSIVE FEATURE COMPARISON OF RELATED WORKS AND SPIDER

Scheme	PUF	PQC	Fine-Grained AC	Revocation	TEE Support	Fog/Edge	Load Balancing
[4]	×	✓	✓	✓	×	✓	×
[5]	×	×	✓	✓	×	✓	×
[22]	×	×	×	×	×	✓	✓
[34]	×	✓	×	×	✓	×	×
[35]	×	✓	×	×	×	×	✓
[36]	×	✓	×	×	×	×	×
[37]	×	×	×	×	×	✓	✓
[39]	×	×	×	×	×	✓	✓
[40]	×	×	×	×	×	✓	✓
(Ours)	✓	✓	✓	✓	✓	✓	✓

protection, and TEE-assisted secure processing, interact with adaptive load balancing in heterogeneous fog environments. This gap motivates the need for a framework that jointly considers security overhead and runtime scheduling efficiency in fog-assisted IIoT systems.

III. OUR PROPOSED SYSTEM

This section presents the proposed post-quantum fog-assisted security architecture for the Industrial Internet of Things (IIoT), enhanced with a novel Spider-based intelligent load balancing mechanism.

A. System Model

As illustrated in Fig. 1, the proposed system adopts a layered architecture consisting of four layers: *Edge Layer*, *Fog Layer*, *Cloud Layer*, and *Secure Layer*. A key feature of the design is the integration of the **Spider Scheduler** within the fog layer, enabling security-aware, capability-aware, and *hierarchical enclave-aware* workload orchestration across distributed fog resources.

1) Layered Architecture Overview:

- **Edge Layer:** Data generation, device authentication, and initial aggregation.
- **Fog Layer:** Secure processing and Spider-based workload scheduling.
- **Cloud Layer:** Storage of encrypted IIoT data and meta-data.
- **Secure Layer:** Key management and access control via the Attribute Authority.

2) *System Entities and Spider Integration:* The system consists of the following components:

- 1) **Attribute Authority (AA)** initializes the lattice-based CP-ABE scheme, issues attribute keys, and defines access policies.
- 2) **IIoT Devices** are resource-constrained nodes equipped with PUFs for hardware-rooted identity. They perform lightweight authenticated encryption before transmission.
- 3) **Edge Gateway (GW)** authenticates devices via CRP validation, verifies packet integrity, and aggregates packets into batches

$$B_k = \{p_1, \dots, p_n\}$$

which are forwarded to the fog layer.

- 4) **CRP Database Server** stores challenge-response pairs for PUF-based authentication.
- 5) **Master Fog Node (MFN) with Spider Controller** orchestrates scheduling via the **Spider Scheduler**, which performs multi-factor routing based on queue state, EPC availability, latency, workload intensity, node capability, and trust condition. The engine includes batch profiling, state monitoring, score computation, node selection, and secure delegation.
- 6) **Fog Nodes (FN)** are TEE-enabled servers hosting multiple enclaves

$$F_j = \{E_{j,k}\}_{k=1}^{m_j}$$

Each enclave operates independently with local state (queue, service rate, EPC).

Fog nodes follow a **dual execution model**:

- **TEE:** Secure operations (key handling, aggregation, partial CP-ABE)
- **REE:** Non-sensitive tasks (policy expansion, formatting)

Spider performs **hierarchical scheduling**:

- Inter-node (across fog nodes)
- Intra-node (across enclaves)

A bounded resource model ensures stable operation under EPC constraints.

- 7) **Spider-Based Secure Delegation** supports workload migration using post-quantum encapsulation (e.g., Kyber) to protect intermediate states.
- 8) **Cloud Storage** is honest-but-curious and stores only encrypted data.
- 9) **Authorized Users (AUs)** decrypt data if their attributes satisfy CP-ABE policies.

B. Threat Model

We consider a strong PPT adversary with full control over public communication channels in the IIoT environment, targeting devices, networks, fog infrastructure, and cloud storage.

Adversary Capabilities. The adversary can observe, inject, modify, replay, and delay messages, and may compromise components outside trusted execution boundaries.

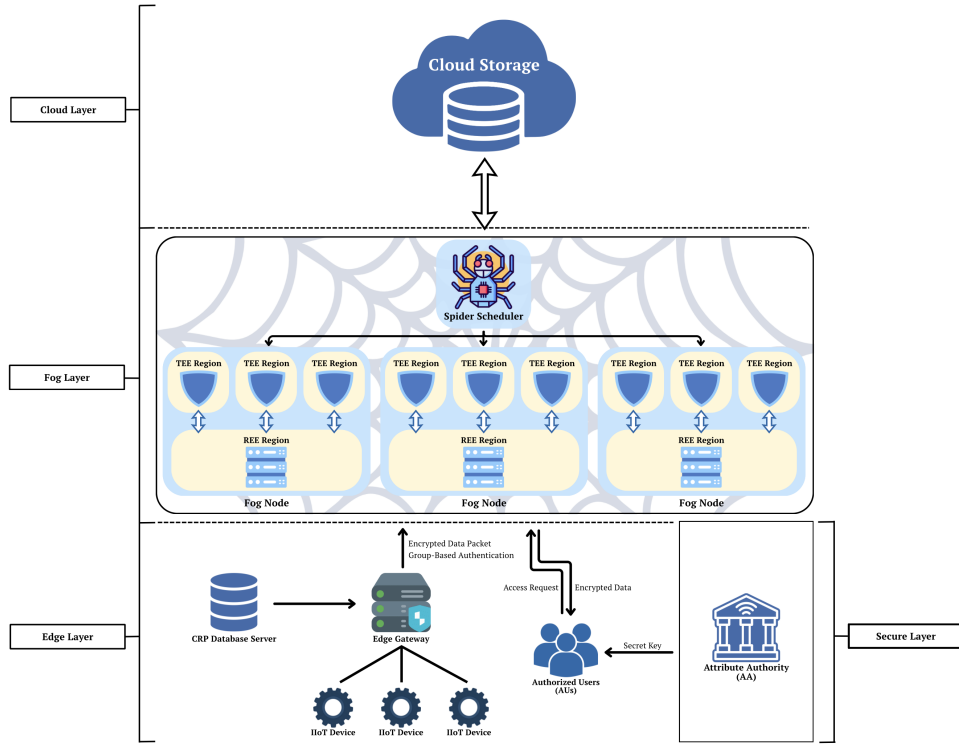


Fig. 1. Spider-Enabled Fog Computing Architecture for Secure IIoT Data Processing

- **Device Attacks:** Impersonation, data forgery, and replay of captured transmissions.
- **Network Adversary:** Full control of communication links, enabling eavesdropping, modification, replay, delay, and DoS attacks.
- **Compromised Fog (REE):** The REE may be fully compromised, allowing memory inspection and execution manipulation, while TEE preserves enclave confidentiality and integrity.
- **Faulty/Malicious Fog Nodes:** Nodes may produce incorrect results or delays due to faults or overload; Spider mitigates this via trust- and capability-aware scheduling.
- **Scheduler Attacks:** Injection of false node metrics or workload manipulation to disrupt scheduling decisions.
- **TEE Side-Channel Attacks:** Leakage via timing or memory access patterns; partially mitigated but not eliminated.
- **Unauthorized Access:** Adversaries attempt to bypass CP-ABE policies to access encrypted cloud data.
- **Data Integrity Attacks:** Tampering with ciphertext, metadata, or intermediate results.
- **Quantum Adversary:** Capable of breaking classical cryptography; security relies on post-quantum assumptions.

C. System Process

The proposed scheme operates through seven phases: system initialization, hardware-bound IIoT encryption, gateway validation and micro-batch formation, capability-

aware leader election, dynamic load balancing, TEE-assisted split-phase CP-ABE processing, and authorized user decryption.

Phase I: System Initialization: The Attribute Authority (AA) initializes the lattice-based CP-ABE system by generating public parameters and master secret keys. To preserve semantic privacy against untrusted host environments, the AA defines a universe of attributes Ω and a collision-resistant hash function $\mathcal{H} : \{0, 1\}^* \rightarrow \mathbb{Z}_q$. For each semantic attribute $\text{attr}_j \in \Omega$, the AA computes a deterministic identifier:

$$ID_j = \mathcal{H}(\text{attr}_j) \quad (1)$$

The AA executes lattice trapdoor generation to produce the Master Public Key (MPK) and Master Secret Key (MSK), binding public parameter matrices to the hashed identifiers:

$$(MPK, MSK) \leftarrow \text{Setup}(1^\lambda, \{ID_j\}_{j=1}^n) \quad (2)$$

To eliminate repeated policy expansion overhead, the access structure is precomputed during system initialization. Specifically, the policy tree TID is transformed into its Linear Secret Sharing Scheme (LSSS) representation and cached for reuse.

$$(M, \rho_{ID}) \leftarrow \text{PolicyTreeGen}(TID) \quad (3)$$

$$\mathcal{C}_{policy}(TID) = (M, \rho_{ID}) \quad (4)$$

Authorized users are issued attribute-linked secret keys:

$$SK_u \leftarrow \text{KeyGen}(MSK, \{ID_j\}_{j \in \text{Attr}_u}) \quad (5)$$

Gateway Initialization The gateway generate a CRYSTALS-Kyber key pair as follow:

$$(pk_{GW}, sk_{GW}) \leftarrow \text{KeyGen}_{Kyber}(1^\lambda) \quad (6)$$

where pk_{GW} denotes the public key, sk_{GW} denotes the corresponding secret key, 1^λ represents the security parameter of length λ , and KeyGen_{Kyber} is the key generation algorithm of the CRYSTALS-Kyber scheme.

The IIoT device derive a secret key and cipherkey using post-quantum key encapsulation every:

$$(cg_{kem,i}, kg_{kem,i}) \leftarrow \text{Encap}_{Kyber}(pk_{GW}) \quad (7)$$

Fog Node Initialization Each fog node initializes its post-quantum communication capability by generating a CRYSTALS-Kyber key pair:

$$(pk_{FN}, sk_{FN}) \leftarrow \text{KeyGen}_{Kyber}(1^\lambda) \quad (8)$$

The public key pk_{FN} is distributed to registered IIoT devices for secure key establishment.

Kyber Precomputation To accelerate lattice-based operations, each fog node precomputes the expanded public matrix and its Number Theoretic Transform (NTT) representation. This enables efficient polynomial operations during encapsulation and decapsulation.

$$A \leftarrow \text{ExpandA}(seed_{FN}) \quad (9)$$

$$\hat{A} = \text{NTT}(A) \quad (10)$$

$$\mathcal{C}_{Kyber}(F_j) = \hat{A} \quad (11)$$

In practical deployments, cached cryptographic structures are not uniformly distributed across fog nodes. Therefore, we assume that only a subset of nodes maintains precomputed policy and Kyber components, reflecting realistic workload locality.

To support hierarchical scheduling in Spider, each fog node initializes multiple TEE enclaves for parallel secure execution and local workload isolation. and initializes local telemetry monitors to track enclave-level states, including queue length, service rate, and EPC availability. These metrics are periodically reported to the Master Fog Node (MFN) to enable global scheduling decisions.

Policy Preparation IIoT devices (or associated gateways) pre-compute hashed attribute identifiers and construct the access policy tree $\mathbb{T}_{ID}$ using $\{ID_j\}$, ensuring that no semantic attribute information is exposed during encryption and evaluation.

Phase II: Hardware-Bound IIoT Encryption: When device ID_i senses data m_i , it first derives a hardware-bound secret using a Physical Unclonable Function (PUF). A noisy response is obtained and reconstructed into a stable secret via a fuzzy extractor:

$$R_{noisy,i} \leftarrow f_{PUF}(C_i) \quad (12)$$

$$R_{secret,i} \leftarrow \text{FuzzyRep}(R_{noisy,i}, HD_i) \quad (13)$$

The device then performs post-quantum key encapsulation to the fog node public key:

$$(c_{kem,i}, k_{kem,i}) \leftarrow \text{Encap}_{Kyber}(pk_{FN}) \quad (14)$$

A device-bound session key is derived as:

$$K_{S,i} = \text{KDF}(H(k_{kem,i}) \parallel R_{secret,i} \parallel ID_i \parallel t_i \parallel ctr_i) \quad (15)$$

The associated authenticated data (AAD) is constructed as:

$$AAD_i = ID_i \parallel t_i \parallel ctr_i \quad (16)$$

The sensed data is encrypted using ChaCha20-Poly1305:

$$(CT_i, Tag_i) \leftarrow \text{ChaCha20-Poly1305.Enc}(K_{S,i}, N_i, m_i, AAD_i) \quad (17)$$

To enable gateway-side packet authentication and early filtering, the device computes a message authentication code using $kg_{kem,i}$:

$$Auth_i \leftarrow \text{MAC}_{kg_{kem,i}}(ID_i \parallel Metadata \parallel c_{kem,i} \parallel CT_i \parallel Tag_i \parallel t_i) \quad (18)$$

Finally, the device transmits the packet to the gateway:

$$P_i = \{ID_i, Metadata, c_{kem,i}, cg_{kem,i}, CT_i, N_i, AAD_i, Tag_i, Auth_i\} \quad (19)$$

Phase III: Gateway Validation and Micro-Batch Formation: Upon receiving packets from IIoT devices, the gateway performs authentication and integrity verification to filter out invalid or malicious inputs before forwarding data to the fog layer.

For each received packet P_i , the gateway recomputes the message authentication code using the derived kyber key:

$$kg_{kem,i} \leftarrow \text{Decap}_{Kyber}(cg_{kem,i}) \quad (20)$$

$$Auth'_i \leftarrow \text{MAC}_{kg_{kem,i}}(ID_i \parallel Metadata \parallel c_{kem,i} \parallel CT_i \parallel Tag_i \parallel t_i) \quad (21)$$

The packet is accepted only if $Auth'_i = Auth_i$. Otherwise, it is discarded. This ensures that only packets originating from legitimate devices with valid session keys are admitted into the system.

All validated packets collected within a fixed time window t_{window} are grouped into a micro-batch. Let $\mathcal{P}_{valid}$ denote the set of validated packets. To ensure batch-level integrity and prevent tampering, the gateway computes a group authentication tag:

$$GTag \leftarrow H(\text{Sort}(Auth_i \mid P_i \in \mathcal{P}_{valid}) \parallel t_G) \quad (22)$$

where $\text{Sort}(\cdot)$ enforces deterministic ordering and t_G is the gateway timestamp.

The micro-batch is then constructed as:

$$B_k = (BID, \mathcal{P}_{valid}, GTag, t_G) \quad (23)$$

The validated batch B_k is forwarded to the Master Fog Node (MFN) for scheduling under Spider.

Remark: Under the hierarchical scheduling model introduced in Phase IV, the batch B_k may be processed as a whole

Algorithm 1 Gateway Validation and Micro-Batch Formation**Require:** Incoming packet stream during epoch t_{window} **Ensure:** Validated micro-batch B_k

```

1:  $\mathcal{P}_{valid} \leftarrow \emptyset$ 
2:  $\mathcal{M} \leftarrow \emptyset$ 
3: for all received packets  $P_i$  do
4:   Compute:
5:    $kg_{kem,i} \leftarrow Decap_{Kyber}(cg_{kem,i})$ 
6:    $Auth'_i \leftarrow MAC_{kg_{kem,i}} \left( ID_i \parallel Metadata \parallel c_{kem,i} \parallel \begin{pmatrix} CT_i \\ Tag_i \parallel t_i \end{pmatrix} \right)$ 
7:   if  $Auth'_i = Auth_i$  then
8:      $\mathcal{P}_{valid} \leftarrow \mathcal{P}_{valid} \cup \{P_i\}$ 
9:      $\mathcal{M} \leftarrow \mathcal{M} \cup \{Auth_i\}$ 
10:  else
11:    Discard  $P_i$ 
12:  end if
13: end for
14: if  $\mathcal{P}_{valid} = \emptyset$  then
15:   return  $\emptyset$ 
16: end if
17:  $t_G \leftarrow GatewayTimestamp()$ 
18: Compute:  $[GTag \leftarrow H(\text{Sort}(\mathcal{M}) \parallel t_G)]$ 
19:  $B_k \leftarrow BID, \mathcal{P}_{valid}, GTag, t_G$ 
20: return  $B_k$ 

```

or decomposed into sub-batches $B_k^{(1)}, \dots, B_k^{(r)}$ to enable enclave-parallel execution.

The complete validation and micro-batch formation procedure is summarized in Algorithm 1.

Phase IV: Spider Hierarchical Enclave-Oriented Load Balancing: This phase presents **Spider** (Secure Priority-Aware Intelligent Dual-Level Enclave Routing), a hierarchical scheduling framework for security-aware and enclave-parallel workload orchestration in heterogeneous fog-assisted IIoT environments.

Unlike conventional fog schedulers that treat each fog node as a single execution entity, Spider explicitly models multiple parallel TEEs within each fog node and performs: (i) *inter-node scheduling* across fog nodes, and (ii) *intra-node enclave scheduling* across local enclaves.

This hierarchical design improves secure workload distribution, mitigates EPC contention, and enables adaptive enclave-parallel execution under dynamic IIoT workloads.

1) *Hierarchical Resource Model:* Each fog node is modeled as:

$$F_j = \{E_{j,k} \mid k = 1, \dots, m_j\} \quad (24)$$

where m_j denotes the number of enclaves hosted by fog node F_j .

Each enclave maintains:

$$\Psi(E_{j,k}) = (q_{j,k}, \mu_{j,k}, M_{j,k}^{free}, \rho_{j,k}) \quad (25)$$

where $q_{j,k}$ denotes queue length, $\mu_{j,k}$ denotes average service rate, $M_{j,k}^{free}$ denotes available EPC memory, and $\rho_{j,k}$ captures enclave contention and paging intensity.

The normalized EPC availability ratio is:

$$\epsilon_j = \frac{M_j^{free}}{M_j^{total}} \quad (26)$$

where M_j^{free} and M_j^{total} denote available and total EPC memory, respectively.

The capability score is defined as:

$$C_j = \alpha_c CPU_j + \beta_c MEM_j + \gamma_c BW_j \quad (27)$$

where CPU_j denotes the normalized processing capability of fog node F_j , MEM_j denotes normalized memory availability, BW_j denotes normalized network bandwidth capacity.

The runtime trust-health score is defined as:

$$U_j = w R_{attest}(F_j) + w_b R_{integ}(F_j) + w_c (1 - A_{anomaly}(F_j)) \quad (28)$$

where $R_{attest}(F_j)$ denotes as a ratio of successful enclave attestations, R_{integ} denotes integration verifying pass rate, $A_{anomaly}$ denotes as the abnormal event rate

To capture the nonlinear behavior of EPC paging overhead under enclave saturation, Spider models EPC pressure as:

$$\rho_{epc}(F_j) = \frac{1}{1 + e^{-\kappa(\epsilon_j - \tau_{epc})}} \quad (29)$$

where τ_{epc} denotes the EPC saturation threshold and κ controls overload sensitivity.

2) *Batch Profiling:* Upon receiving a validated batch B_k , the Master Fog Node (MFN) constructs:

$$\Phi(B_k) = (S_k, \omega_k, \eta_k, \delta_k) \quad (30)$$

where S_k denotes batch size, ω_k denotes workload intensity, η_k denotes security urgency, and δ_k denotes deadline sensitivity.

Spider estimates workload intensity using normalized workload and enclave-related factors:

$$\omega_k = \beta_1 \hat{S}_k + \beta_2 \widehat{Attr}_k + \beta_3 \widehat{Depth}_k + \beta_4 \widehat{EPC}_k \quad (31)$$

where $\hat{S}_k$, $\widehat{Attr}_k$, $\widehat{Depth}_k$, and $\widehat{EPC}_k$ denote normalized batch size, attribute complexity, policy depth, and estimated enclave-memory pressure, respectively.

The urgency score is:

$$\eta_k = \gamma_1 \frac{N_{susp}(B_k)}{|B_k|} + \gamma_2 AnomalyFlag(B_k) \quad (32)$$

where $N_{susp}(B_k)$ denotes suspicious packets detected during gateway validation. **AnomalyFlag is boolean value denote that this batch has anomaly or bit error?**

Deadline urgency is normalized as:

$$\delta_k = 1 - \frac{D_k - t_{now}}{D_{max}} \quad (33)$$

D_k, D_{max} denotes as deadline of this batch and all deadline?

3) *Fog Node State Model*: Each fog node periodically reports:

$$\Psi(F_j) = (Q_j^{TEE}, Q_j^{REE}, \mu_j^{TEE}, \mu_j^{REE}, L_j, \epsilon_j, C_j, U_j) \quad (34)$$

where Q_j^{TEE} and Q_j^{REE} denote queue lengths, μ_j^{TEE} and μ_j^{REE} denote average service rates, L_j denotes network latency, ϵ_j denoted as EPC availability ratio, C_j denotes capability score, and U_j denotes runtime trust-health score.

Telemetry collection is performed asynchronously using lightweight epoch-based monitoring to minimize scheduling overhead.

Level 1: Inter-Node Scheduling: Spider selects the optimal fog node by minimizing a multi-factor scheduling score.

The waiting-time approximation is:

$$T_{wait}(F_j) = \frac{Q_j^{TEE} + 1}{\mu_j^{TEE}} + \frac{Q_j^{REE} + 1}{\mu_j^{REE}} \quad (35)$$

which assumes bounded workload variance within each scheduling epoch.

The EPC pressure penalty is:

$$P_{epc}(F_j, B_k) = \rho_{epc}(F_j) \frac{M_k^{req}}{M_j^{free}} \quad (36)$$

The enclave memory requirement M_k^{req} is estimated using an offline-calibrated cryptographic cost model.

The capability and trust penalties are:

$$P_{cap}(F_j, B_k) = \max(0, \omega_k - C_j) \quad (37)$$

$$P_{trust}(F_j) = (1 - U_j) \quad (38)$$

To reduce redundant computation, Spider prioritizes fog nodes with reusable cached structures:

$$R_{reuse}(F_j, B_k) = \theta_1 I_{policy}(F_j, B_k) + \theta_2 I_{Kyber}(F_j) \quad (39)$$

where I_{policy} and I_{Kyber} denote reusable policy-cache and cryptographic-cache indicators, respectively.

The final scheduling score is:

$$\begin{aligned} SpiderScore(F_j, B_k) = & w_1 T_{wait}(F_j) + w_2 L_j + w_3 P_{epc} \\ & + w_4 P_{cap} + w_5 P_{trust} \\ & - w_6 \eta_k U_j - w_7 \delta_k \mu_j^{TEE} \\ & - w_8 R_{reuse}(F_j, B_k) \end{aligned} \quad (40)$$

The selected fog node is:

$$F^* = \arg \min_{F_j \in F_{feasible}} SpiderScore(F_j, B_k) \quad (41)$$

Level 2: Intra-Node Enclave Scheduling: After selecting F^* , Spider performs enclave-level scheduling.

The enclave waiting-time approximation is:

$$T_{wait}(E_{j,k}) = \frac{q_{j,k} + 1}{\mu_{j,k}} \quad (42)$$

The enclave EPC penalty is:

$$P_{epc}(E_{j,k}, B_k) = \max \left(0, \frac{M_k^{req}}{M_{j,k}^{free}} - \tau \right)^2 \quad (43)$$

Spider estimates workload affinity using weighted overlap among policy structures, attribute distributions, and batch-scale characteristics:

$$A(E_{j,k}, B_k) = \begin{cases} 1, & Sim(B_a, B_k) \geq \tau_{sim} \\ 0, & otherwise \end{cases} \quad (44)$$

where $Sim(\cdot)$ denotes workload similarity estimation, τ_{sim} denotes threshold of $Sim(\cdot)$.

The enclave scheduling score is:

$$\begin{aligned} EnclaveScore(E_{j,k}, B_k) = & z_1 T_{wait}(E_{j,k}) \\ & + z_2 P_{epc}(E_{j,k}, B_k) \\ & + z_3 p_{j,k} \\ & - z_4 A(E_{j,k}, B_k) \end{aligned} \quad (45)$$

The selected enclave is:

$$E^* = \arg \min_{E_{j,k} \in F^*} EnclaveScore(E_{j,k}, B_k) \quad (46)$$

4) *Parallel Batch Decomposition*: To exploit enclave-level parallelism, Spider decomposes:

$$B_k \rightarrow \{B_k^{(1)}, \dots, B_k^{(r)}\} \quad (47)$$

where r is adaptively determined according to enclave availability, queue state, EPC capacity, and workload intensity.

Spider assumes cryptographic independence among sub-batches and enforces deterministic merge ordering during secure aggregation.

Each sub-batch is independently scheduled:

$$E^{*(i)} = \arg \min EnclaveScore(E_{j,k}, B_k^{(i)}) \quad (48)$$

This enables concurrent cryptographic execution while reducing enclave contention and end-to-end latency.

5) *Admission and Stability Control*: A task is admitted only if:

$$M_{j,k}^{free} \geq \alpha M_k^{req} \quad (49)$$

where $\alpha > 1$ denotes a conservative EPC safety margin.

To reduce scheduling oscillation under bursty workloads, Spider applies exponential score smoothing:

$$\bar{S}_j(t) = \alpha_s S_j(t) + (1 - \alpha_s) \bar{S}_j(t-1) \quad (50)$$

A scheduling update is triggered only if:

$$|\bar{S}_j(t) - \bar{S}_j(t-1)| \geq \zeta \quad (51)$$

6) *Secure Delegation*: If no feasible enclave exists, Spider performs secure delegation using lightweight continuation metadata:

$$\Xi_k = (ID_k, Meta_k, CT_k^{partial}, A_k, T_k, epoch_k, \sigma_k) \quad (52)$$

Spider delegates only non-sensitive continuation states rather than full enclave memory images, thereby reducing migration overhead and minimizing enclave-state exposure. The epoch identifier and signature prevent stale-state replay and unauthorized delegation injection.

Delegation confidentiality is protected using Kyber-based post-quantum encapsulation.

The overall scheduling complexity is:

$$\mathcal{O}\left(|\mathcal{F}| + \sum_j m_j\right) \quad (53)$$

Spider therefore transforms conventional fog scheduling into a hierarchical enclave-oriented orchestration framework that jointly optimizes secure throughput, EPC utilization, workload stability, and latency-aware execution for large-scale post-quantum IIoT environments.

Phase V: Multi-Enclave TEE-Assisted Split-Phase CP-ABE Processing: After receiving the validated batch B_k , the selected fog node F^* performs secure enclave-parallel processing using a two-level architecture: (i) enclave-parallel authenticated aggregation and (ii) split-phase lattice-based CP-ABE encryption.

Unlike conventional fog-assisted aggregation approaches that expose intermediate plaintext aggregation to the untrusted runtime environment, the proposed design ensures that all plaintext processing, chunk encryption, key derivation, and aggregation authentication remain strictly inside TEE boundaries. The REE only handles encrypted aggregation chunks and blinded CP-ABE expansion components, thereby preventing replay, reordering, omission, duplication, and manipulation of enclave outputs.

Step 1: Batch Integrity Verification (REE): The fog node first verifies the integrity of the gateway-formed batch by recomputing:

$$GTag' \leftarrow H(\text{Sort}(\{Auth_i \mid P_i \in \mathcal{P}_{valid}\}) \parallel t_G) \quad (54)$$

The batch is accepted only if:

$$GTag' = GTag \quad (55)$$

Otherwise, the batch is discarded.

Step 2: Secure Parallel Batch Decomposition: Based on Spider scheduling, the validated batch is either processed as a whole or decomposed into enclave-parallel sub-batches:

$$B_k \rightarrow \{B_k^{(1)}, \dots, B_k^{(r)}\} \quad (56)$$

where r denotes the enclave-parallel decomposition factor.

Unlike static partitioning approaches, Spider dynamically determines r according to:

- Available enclave instances
- EPC memory availability
- Current enclave queue occupancy
- Batch size
- Estimated cryptographic processing cost

Let:

- $|B_k|$ denote the number of packets in the batch
- $\bar{s}_k$ denote the average packet size
- M_{EPC} denote available EPC memory
- Q_j denote enclave queue occupancy
- N_E denote the number of available enclaves
- α_{enc} denote estimated encryption cost per packet

- α_{agg} denote aggregation cost per packet

Spider first estimates the expected enclave workload:

$$\mathcal{W}(B_k) = |B_k| \cdot (\alpha_{enc} + \alpha_{agg}) \quad (57)$$

To prevent EPC overflow during enclave-resident aggregation, Spider constrains the maximum sub-batch size:

$$|B_k^{(i)}| \cdot \bar{s}_k \leq \eta M_{EPC} \quad (58)$$

where:

$$0 < \eta < 1$$

is a safety factor reserving EPC memory for enclave runtime, metadata buffers, and cryptographic state.

The decomposition factor is then computed as:

$$r = \min\left(N_E, \left\lceil \frac{|B_k| \cdot \bar{s}_k}{\eta M_{EPC}} \right\rceil + \left\lceil \frac{\mathcal{W}(B_k)}{\Gamma_{max}} \right\rceil + Q_j\right) \quad (59)$$

where Γ_{max} denotes the maximum sustainable cryptographic workload per enclave during one scheduling interval.

If:

$$r = 1 \quad (60)$$

Spider processes the batch sequentially within a single enclave to avoid unnecessary orchestration overhead.

Otherwise, Spider partitions the batch into r approximately balanced sub-batches:

$$|B_k^{(i)}| \approx \frac{|B_k|}{r} \quad (61)$$

To guarantee exact-once processing under parallel execution, Spider enforces:

$$\bigcup_{i=1}^r B_k^{(i)} = B_k \quad (62)$$

$$B_k^{(i)} \cap B_k^{(j)} = \emptyset, \quad \forall i \neq j \quad (63)$$

Each sub-batch is assigned:

$$SubID_i = H(BID \parallel i \parallel epoch_k) \quad (64)$$

and dispatched to enclave $E^{*(i)}$

This adaptive decomposition strategy enables Spider to:

- Prevent EPC paging under high-volume IIoT streams
- Balance cryptographic workloads across enclaves
- Minimize aggregation latency
- Avoid enclave oversubscription
- Support scalable enclave-parallel secure aggregation

Step 3: Enclave-Parallel Secure Aggregation and Chunk Encryption (TEE): A fresh master aggregation key is first generated inside the trusted enclave:

$$K_{AES}^{master} \leftarrow \{0, 1\}^{256} \quad (65)$$

Each enclave independently processes its assigned sub-batch. For every packet $P_\ell \in B_k^{(i)}$, the enclave decrypts:

$$m_\ell \leftarrow \text{ChaCha20-Poly1305.Dec}(K_{S,\ell}, N_\ell, CT_\ell, AAD_\ell, Tag_\ell) \quad (66)$$

Ciphertexts failing integrity verification are immediately discarded.

To reduce EPC pressure under large-scale IIoT streams, Spider performs streaming enclave aggregation instead of materializing the entire plaintext batch simultaneously inside EPC memory.

The enclave incrementally updates:

$$S_j = H(S_{j-1} \parallel m_j) \quad (67)$$

where S_j denotes the progressive aggregation state.

The enclave derives a deterministic chunk-encryption key:

$$K_{chunk}^{(i)} = \text{KDF}(K_{AES}^{master} \parallel BID \parallel SubID_i \parallel epoch_k) \quad (68)$$

A fresh initialization vector is generated:

$$IV_i \leftarrow \{0, 1\}^{96} \quad (69)$$

The enclave incrementally encrypts the local aggregation chunk:

$$(C_i, Tag_i) \leftarrow \text{AES-GCM.Enc}(K_{chunk}^{(i)}, IV_i, \mathcal{M}_i, AAD_i) \quad (70)$$

where $\mathcal{M}_i$ denotes the enclave-local streaming aggregation chunk and AAD_i denotes the corresponding authenticated metadata stream.

To cryptographically bind the enclave output to the execution context, the enclave computes:

$$h_i = H(BID \parallel SubID_i \parallel epoch_k \parallel C_i \parallel Tag_i \parallel IV_i \parallel AAD_i) \quad (71)$$

The enclave further generates an authenticated execution token:

$$\tau_i = \text{MAC}_{K_i^{TEE}}(h_i) \quad (72)$$

where K_i^{TEE} is an enclave-sealed authentication key established during enclave initialization or attestation.

The enclave outputs:

$$\Gamma_i = (SubID_i, C_i, Tag_i, IV_i, AAD_i, h_i, \tau_i) \quad (73)$$

Importantly, plaintext aggregation states never leave the TEE boundary.

Step 4: Trusted Deterministic Merge and Aggregation Commitment: After enclave processing completes, a trusted merger enclave verifies each authenticated enclave output:

$$\text{VerifyMAC}_{K_i^{TEE}}(h_i, \tau_i) \quad (74)$$

Invalid enclave outputs are rejected immediately.

To eliminate nondeterministic aggregation behavior under parallel execution, Spider applies deterministic merge ordering:

$$\mathcal{O} = \text{SortBy}(SubID_i) \quad (75)$$

Encrypted aggregation chunks are merged according to $\mathcal{O}$:

$$CT_{AES} =_{\Gamma_i \in \mathcal{O}} C_i \quad (76)$$

Similarly:

$$Tag_{AES} =_{\Gamma_i \in \mathcal{O}} Tag_i \quad (77)$$

$$AAD_{agg} =_{\Gamma_i \in \mathcal{O}} AAD_i \quad (78)$$

The aggregation initialization-vector set is:

$$\mathcal{IV} = \{IV_i\}_{i=1}^r \quad (79)$$

Let:

$$(h_{(1)}, \dots, h_{(r)}) \quad (80)$$

denote the enclave digests ordered according to $\mathcal{O}$.

Spider computes the global aggregation commitment:

$$Root_k = H(BID \parallel epoch_k \parallel r \parallel h_{(1)} \parallel h_{(2)} \parallel \dots \parallel h_{(r)}) \quad (81)$$

Since the digest sequence follows deterministic ordering, any replay, duplication, omission, or reordering changes $Root_k$.

Therefore, $Root_k$ acts as a lightweight authenticated aggregation commitment over all enclave-parallel outputs.

Step 5: Partial CP-ABE Encryption within TEE: To protect the master aggregation key under the access policy, the enclave initiates lattice-based CP-ABE encryption.

The enclave samples:

$$s, \mathbf{e}_0, \mathbf{e}_{base} \leftarrow \chi \quad (82)$$

and constructs:

$$\mathbf{v} = (s, y_2, \dots, y_n)^\top \quad (83)$$

The core ciphertext component is:

$$CT_0 = \mathbf{A}^\top s + \mathbf{e}_0 + K_{AES}^{master} \left\lfloor \frac{q}{2} \right\rfloor \pmod{q} \quad (84)$$

To enable efficient REE-side policy expansion, the enclave constructs:

$$\mathbf{V}_{base} = \mathcal{C}_{base}(TID)\mathbf{v} + \mathbf{e}_{base} \quad (85)$$

Since:

$$\mathcal{C}_{base}(TID) = \mathbf{A}^\top \quad (86)$$

is independent of runtime randomness, Spider precomputes and caches the matrix transpose per access policy.

Only blinded non-sensitive components are exported:

$$\Psi_{out} = (CT_{AES}, Tag_{AES}, \mathcal{IV}, CT_0, \mathbf{V}_{base}, TID, Root_k) \quad (87)$$

Sensitive values including:

$$s, \mathbf{v}, K_{AES}^{master}, K_{chunk}^{(i)}, \mathcal{M}_i$$

remain strictly inside the enclave and are erased after execution.

Step 6: CP-ABE Completion in REE: The REE completes only the non-sensitive policy expansion.

The cached LSSS representation is retrieved:

$$(M, \rho_{ID}) \leftarrow \mathcal{C}_{policy}(TID) \quad (88)$$

For each row:

$$CT_i = M_i \mathbf{V}_{base} + \mathbf{A}_{\rho_{ID}(i)}^\top \mathbf{r}_i + \mathbf{e}_{1,i} \pmod{q} \quad (89)$$

The final CP-ABE ciphertext becomes:

$$CT_{L-ABE} = (CT_0, (M, \rho_{ID}), \{CT_i\}_{i=1}^\ell) \quad (90)$$

This split-phase design confines all sensitive randomness, plaintext aggregation, and key derivation operations within TEE boundaries while allowing the REE to perform only blinded matrix expansion.

Step 7: Signature and Secure Upload: The fog node signs the encrypted aggregation object together with the aggregation commitment:

$$\sigma \leftarrow \text{Sign}_{\text{Dilithium}}\left(sk_{FN}, H(BID \parallel CT_{AES} \parallel Tag_{AES} \parallel \mathcal{IV} \parallel CT_{L-ABE} \parallel Root_k)\right) \quad (91)$$

The final uploaded object is:

$$\Omega = (BID, CT_{AES}, Tag_{AES}, \mathcal{IV}, CT_{L-ABE}, Root_k, \sigma) \quad (92)$$

Final Remark: Although processing is parallelized across multiple enclaves, Spider outputs a single deterministic and verifiable ciphertext object Ω . Through exact partition constraints, streaming EPC-aware aggregation, enclave-authenticated chunk generation, deterministic merge ordering, and aggregation commitment binding, the proposed design prevents replay, duplication, omission, reordering, and REE-side manipulation while preserving scalable enclave-parallel secure processing for large-scale post-quantum IIoT environments.

Phase VI: User Verification and Decryption: An authorized user retrieves the encrypted object Ω from cloud storage:

$$\Omega = (BID, epoch_k, CT_{AES}, \mathcal{D}_k, CT_{L-ABE}, Root_k, \sigma) \quad (93)$$

where:

$$\mathcal{D}_k = \{(SubID_i, |C_i|, AAD_i, IV_i, Tag_i)\}_{i=1}^r \quad (94)$$

denotes the authenticated chunk manifest containing chunk ordering, ciphertext boundaries, metadata, initialization vectors, and authentication tags.

The user then performs integrity verification, policy validation, CP-ABE decryption, aggregation verification, chunk-key reconstruction, and deterministic plaintext recovery.

Step 1: Integrity and Authenticity Verification: The user first verifies the integrity and origin of the encrypted object using the fog node's public verification key:

$$\text{Verify}_{\text{Dilithium}}\left(pk_{FN}, \sigma, H(BID \parallel epoch_k \parallel CT_{AES} \parallel \mathcal{D}_k \parallel CT_{L-ABE} \parallel Root_k)\right) \quad (95)$$

If verification fails, the ciphertext is rejected immediately.

This step guarantees that the encrypted aggregation chunks, chunk metadata, initialization vectors, authentication tags, and aggregation commitment have not been modified, replayed, reordered, duplicated, or partially omitted.

Step 2: Policy Satisfaction Check: Let:

$$(M, \rho_{ID}) \quad (96)$$

be the LSSS access structure embedded in CT_{L-ABE}

The user verifies whether the possessed attribute set $Attr_u$ satisfies the access policy.

If satisfied, the user derives reconstruction coefficients $\{\omega_i\}$ such that:

$$\sum_{i \in I} \omega_i M_i = (1, 0, \dots, 0) \quad (97)$$

Otherwise, decryption is aborted.

Step 3: Ciphertext Share Evaluation: Using the attribute-based secret key SK_u , the user evaluates:

$$C'_i \leftarrow \text{Eval}(CT_i, SK_u) \approx M_i \mathbf{V}_{base} + e_{eval,i} \quad (98)$$

This operation reconstructs the blinded secret-sharing components associated with the satisfied access structure.

Step 4: Secret Reconstruction: The user reconstructs the blinded vector:

$$C_{rec} = \sum_{i \in I} \omega_i C'_i \approx \mathbf{A}^\top s + e_{total} \quad (99)$$

Correct decryption succeeds when the aggregated lattice noise remains within the decoding threshold.

Step 5: Master Aggregation Key Recovery: The master aggregation key is recovered as:

$$K_{AES}^{master} \leftarrow \text{Decode}(\lfloor CT_0 - C_{rec} \rfloor_2) \quad (100)$$

This key serves as the root secret used to derive all enclave-generated chunk encryption keys.

Step 6: Chunk Parsing and Aggregation Commitment Verification: Using the authenticated chunk manifest $\mathcal{D}_k$, the user parses:

$$CT_{AES} = C_1 \parallel C_2 \parallel \dots \parallel C_r \quad (101)$$

where chunk boundaries are determined using the recorded ciphertext lengths $|C_i|$.

For each chunk descriptor:

$$(SubID_i, |C_i|, AAD_i, IV_i, Tag_i) \in \mathcal{D}_k \quad (102)$$

the user recomputes:

$$h_i = H(BID \parallel SubID_i \parallel epoch_k \parallel C_i \parallel Tag_i \parallel IV_i \parallel AAD_i) \quad (103)$$

Let:

$$(h_{(1)}, \dots, h_{(r)}) \quad (104)$$

denote the digest sequence ordered according to:

$$\mathcal{O} = \text{SortBy}(SubID_i) \quad (105)$$

The user recomputes:

$$Root'_k = H(BID \parallel epoch_k \parallel r \parallel h_{(1)} \parallel h_{(2)} \parallel \dots \parallel h_{(r)}) \quad (106)$$

The encrypted aggregation object is accepted only if:

$$Root'_k = Root_k \quad (107)$$

This step guarantees aggregation completeness and prevents chunk replay, duplication, omission, substitution, and reordering attacks.

Step 7: Chunk Key Derivation and Parallel Decryption:

For each aggregation chunk, the user derives:

$$K_{chunk}^{(i)} = \text{KDF}(K_{AES}^{master} \parallel BID \parallel SubID_i \parallel epoch_k) \quad (108)$$

The user then decrypts:

$$\mathcal{M}_i \leftarrow \text{AES-GCM.Dec}(K_{chunk}^{(i)}, IV_i, C_i, AAD_i, Tag_i) \quad (109)$$

Chunks failing integrity verification are discarded immediately.

Since chunk decryption is independent, Spider supports parallel client-side decryption across multiple processing cores or trusted execution contexts.

Step 8: Deterministic Aggregation Recovery: The decrypted aggregation chunks are reordered according to:

$$\mathcal{O} = \text{SortBy}(SubID_i) \quad (110)$$

The original aggregated IIoT plaintext is reconstructed as:

$$\mathcal{M}_{agg} = \mathcal{M}_i \in \mathcal{O} \quad (111)$$

Thus, only users whose attributes satisfy the access policy can recover the complete aggregated IIoT data while preserving aggregation integrity, deterministic ordering, and resistance against replay, omission, duplication, reordering, and ciphertext manipulation attacks.

IV. ENCLAVE-AWARE AND STABILITY-AWARE MASTER FOG NODE ELECTION

To coordinate hierarchical workload scheduling in Spider, we propose an enclave-aware and stability-aware Master Fog Node (MFN) election mechanism. Unlike conventional leader-election schemes that rely only on coarse-grained node metrics such as CPU or memory utilization, the proposed mechanism explicitly incorporates enclave-level workload dynamics, EPC pressure, execution trustworthiness, and temporal stability. This enables Spider to select a reliable control-plane coordinator capable of efficiently managing security-sensitive workloads in heterogeneous fog-assisted IIoT environments.

A. Capability and Security-Aware Scoring

Spider first computes a normalized multi-resource capability score for each fog node F_j .

Each fog node F_j periodically computes a composite capability and security-aware score:

$$\text{Score}(F_j) = \alpha_1 C_j + \alpha_2 M_j - \alpha_3 N_j + \alpha_4 R_j + \alpha_5 T_j \quad (112)$$

where C_j represents capability score, M_j represents available enclave memory resources, N_j represents network latency to neighboring nodes or gateways, R_j denotes node readiness, and T_j captures execution trustworthiness.

The trust component T_j is derived from runtime security indicators such as enclave attestation success rate, integrity verification outcomes, and abnormal execution behavior. By integrating trust-awareness into leader election, Spider prevents control-plane coordination from being assigned to nodes exhibiting unstable or suspicious execution patterns.

B. Enclave-Aware Node Readiness

To accurately capture the runtime condition of each fog node, Spider computes readiness by jointly considering enclave-level and node-level resource conditions:

$$R_j = 1 - \left(\beta_1 \cdot \frac{1}{m_j} \sum_{k=1}^{m_j} \frac{Q_{j,k}}{Q_{\max}} + \beta_2 \cdot \rho_{epc}(F_j) + \beta_3 \cdot \frac{Q_j^{REE}}{Q_{\max}^{REE}} \right) \quad (113)$$

where $Q_{j,k}$ denotes the queue length of enclave $E_{j,k}$, m_j is the number of enclaves hosted by node F_j , $\rho_{epc}(F_j)$ captures EPC memory pressure, and Q_j^{REE} represents backlog within the untrusted execution environment (REE).

The readiness score is normalized such that:

$$0 \leq R_j \leq 1 \quad (114)$$

Unlike traditional node-level monitoring approaches, this formulation explicitly captures fine-grained enclave imbalance and secure-execution contention. Consequently, Spider can more accurately estimate a node's ability to coordinate security-sensitive workloads under dynamic enclave utilization and EPC constraints.

C. Stability-Aware MFN Selection

To avoid frequent leader oscillations under fluctuating workloads, Spider incorporates temporal stability into the leader-election process:

$$MFN = \arg \max_{F_j \in \mathcal{F}_{candidate}} (\text{Score}(F_j) - \gamma \cdot \Delta \text{Score}(F_j)) \quad (115)$$

where $\Delta \text{Score}(F_j)$ captures short-term variations in node capability and runtime stability. The penalty term discourages selecting nodes with unstable performance, thereby improving control-plane continuity and reducing coordination overhead.

Candidate nodes are selected according to the readiness constraint:

$$F_j \in \mathcal{F}_{candidate} \quad \text{if} \quad R_j \geq \tau \quad (116)$$

where τ denotes the minimum readiness threshold required for participating in MFN election.

D. Control-Plane Coordination

After election, the MFN transitions into a dedicated control-plane coordinator responsible for global Spider scheduling, including inter-node workload distribution and intra-node enclave assignment. Residual local workloads may be redistributed to neighboring fog nodes to prevent bottlenecks caused by simultaneous control-plane and data-plane execution.

This separation between coordination and workload execution improves scalability and scheduling responsiveness in large-scale heterogeneous fog environments.

The proposed MFN election mechanism therefore provides a stable and security-aware foundation for coordinating

enclave-parallel workload scheduling in Spider. When MFN degradation or node failure is detected, Spider subsequently invokes the distributed group-based failure detection and secure workload recovery mechanism described in the following section.

V. GROUP-BASED FAILURE DETECTION AND SECURE WORKLOAD RECOVERY

Large-scale heterogeneous fog environments may contain a substantial number of fog nodes operating under dynamic workloads and varying network conditions. In such settings, requiring every fog node to continuously transmit heartbeat messages directly to the Master Fog Node (MFN) introduces excessive control-plane traffic, increases monitoring overhead, and creates scalability bottlenecks under high node density. Moreover, centralized monitoring may degrade responsiveness and increase failure-detection latency when the MFN becomes heavily loaded.

To address these limitations, Spider adopts a decentralized group-based failure detection and secure workload recovery mechanism. Instead of centralized heartbeat collection, fog nodes are partitioned into small neighboring monitoring groups that collaboratively exchange state information and detect failures locally. This design distributes monitoring responsibility across the fog infrastructure, reduces control-plane congestion, and improves scalability in dynamic fog-assisted IIoT environments.

A. Group-Based Collaborative Failure Detection

The fog layer is partitioned into localized monitoring groups:

$$G_i = \{F_1, F_2, \dots, F_s\} \quad (117)$$

where G_i denotes the i -th monitoring group and s denotes the group size satisfying:

$$3 \leq s \leq 7 \quad (118)$$

Small bounded-size groups reduce heartbeat overhead while preserving reliable collaborative failure confirmation. Importantly, the group size does not increase with the total number of fog nodes N . Instead, Spider scales by increasing the number of groups:

$$g = \left\lceil \frac{N}{s} \right\rceil \quad (119)$$

where g denotes the number of monitoring groups. Consequently, the per-node monitoring overhead remains bounded by $O(s)$ while the total monitoring overhead scales linearly with the fog-layer size.

At each monitoring interval t , fog node F_j broadcasts an authenticated heartbeat message to neighboring peers:

$$\text{Heartbeat}(F_j, t) = (ID_j, t, epoch_j, status_j, \sigma_j) \quad (120)$$

where ID_j denotes the fog-node identifier, t represents the heartbeat timestamp, $epoch_j$ identifies the local monitoring

epoch, $status_j$ contains summarized runtime telemetry information, and σ_j denotes an integrity-protection token generated using a post-quantum signature or authenticated MAC.

The authenticated heartbeat prevents forged heartbeat injection, replay manipulation, and malicious state falsification by compromised nodes.

Each neighboring node continuously monitors heartbeat freshness. The elapsed heartbeat interval for fog node F_j is computed as:

$$\Delta t_j = t_{\text{current}} - t_{\text{last}}(F_j) \quad (121)$$

where t_{current} denotes the current monitoring time and $t_{\text{last}}(F_j)$ represents the timestamp of the most recently received heartbeat from node F_j .

A fog node is initially marked as suspicious if:

$$\Delta t_j > \tau_h \quad (122)$$

where τ_h denotes the predefined heartbeat timeout threshold.

To reduce false-positive failure detection caused by transient congestion, communication delay, or temporary scheduling latency, Spider adopts quorum-based collaborative confirmation. Let q denote the minimum number of neighboring peers required to confirm a failure:

$$q \geq \left\lceil \frac{s}{2} \right\rceil \quad (123)$$

A fog node is declared failed only if at least q neighboring nodes independently observe timeout violations. Once confirmed, the failed node is removed from the Spider scheduling pool and excluded from subsequent workload assignment operations.

This decentralized collaborative design eliminates centralized monitoring bottlenecks while improving fault-detection robustness and scalability under dynamic fog conditions.

B. Secure Workload Preservation and Recovery

To preserve workload continuity during node failures, Spider maintains a lightweight secure delegation capsule before enclave execution begins. Unlike conventional checkpointing approaches that replicate complete enclave memory, Spider preserves only essential workload metadata and partially processed cryptographic states required for secure reassignment. This significantly reduces synchronization overhead, communication cost, and enclave-state migration complexity.

The secure delegation capsule is represented as:

$$\Xi_k = (ID_k, SubID_k, Prog_k, Meta_k, CT_k^{\text{partial}}, T_k, epoch_k, \sigma_k) \quad (124)$$

where ID_k denotes the workload identifier, $SubID_k$ denotes the enclave-parallel sub-batch identifier, $Prog_k$ represents workload execution progress, $Meta_k$ contains scheduling metadata, CT_k^{partial} stores partially processed cryptographic state, T_k denotes timestamp information, $epoch_k$ identifies the recovery epoch, and σ_k provides integrity and authenticity protection.

The inclusion of timestamps, epochs, and authenticated integrity protection prevents stale-state replay, unauthorized workload injection, and malicious recovery-state manipulation.

When a fog node failure is confirmed, Spider identifies all unfinished workloads associated with the failed node and extracts only incomplete enclave-parallel sub-batches requiring reassignment. Since Spider already decomposes workloads into enclave-parallel execution units, completed sub-batches remain unaffected while only unfinished fragments are recovered. This significantly reduces recomputation overhead and improves recovery latency.

C. Capability-Aware Recovery Node Selection

After failure confirmation, Spider dynamically selects the most suitable recovery node from the remaining active fog nodes using the existing capability-aware and enclave-aware scheduling framework:

$$F_{recover} = \arg \min_{F_j \in F_{alive}} \text{SpiderScore}(F_j, B_k) \quad (125)$$

where F_{alive} denotes the set of active fog nodes that satisfy scheduling feasibility constraints.

Using the same SpiderScore function for both normal scheduling and failure recovery ensures consistency between workload orchestration and recovery operations. The recovery decision therefore remains jointly optimized according to queue state, enclave availability, EPC pressure, workload intensity, trust condition, and network latency.

After reassignment, the selected recovery node resumes execution from the latest preserved workload state rather than restarting the entire computation pipeline. This minimizes recovery latency, reduces redundant cryptographic processing, and improves service continuity under dynamic node failures.

Therefore, under authenticated heartbeat exchange, quorum-based collaborative failure confirmation, secure delegation preservation, and capability-aware recovery scheduling, Spider ensures that failed fog nodes are safely excluded from scheduling while unfinished workloads are securely reassigned with verifiable state continuity.

Through the integration of decentralized monitoring, enclave-aware workload preservation, and adaptive recovery scheduling, Spider provides scalable, resilient, and security-aware fault tolerance for heterogeneous fog-assisted IIoT environments.

VI. SECURITY ANALYSIS

In this section, we provide a formal security analysis of Spider under a post-quantum adversarial model. The analysis covers confidentiality of outsourced IIoT data, bounded policy privacy, bounded scheduler leakage, deterministic aggregation correctness, replay resistance, end-to-end integrity and authenticity, and forward secrecy of session-level protection.

A. Security Goals

The security objectives of Spider are summarized as follows:

- **Confidentiality of outsourced IIoT data:** An adversary should learn no information about outsourced IIoT plaintext contents beyond explicitly modeled leakage.
- **Policy privacy up to structural leakage:** The adversary may observe the size and structural form of the access policy, but not the semantic meaning of attributes.
- **Bounded scheduler leakage:** Spider should expose only coarse workload abstractions required for dynamic enclave scheduling.
- **Deterministic aggregation correctness:** The reconstructed aggregation must contain every valid sub-batch exactly once while preventing replay, duplication, reordering, or omission.
- **Integrity and authenticity:** Forged packets, modified aggregation chunks, manipulated chunk manifests, or forged fog outputs should be rejected except with negligible probability.
- **Forward secrecy of session protection:** Compromise of later protocol states should not reveal previous device-to-fog session keys or chunk-encryption keys.
- **Post-quantum resistance:** All guarantees must hold against quantum polynomial-time adversaries under the assumed hardness of the underlying post-quantum primitives.

B. Adversarial Model and Trust Assumptions

We consider a quantum polynomial-time adversary $\mathcal{A}$ with full control over public communication channels.

The following trust assumptions are adopted.

- The **Attribute Authority (AA)** is fully trusted and correctly generates public parameters and master secret keys.
- The **cloud** is honest-but-curious: it stores ciphertexts faithfully but may attempt to infer information from encrypted objects and metadata.
- Fog-node **REE environments** are fully untrusted and may attempt replay, duplication, omission, reordering, or aggregation manipulation attacks.
- **TEEs** provide confidentiality and integrity for enclave-resident code, enclave-local memory, sealed keys, and enclave-generated authentication tokens.
- The adversary may observe scheduling metadata, encrypted chunk counts, chunk lengths, and deterministic aggregation ordering, but cannot access enclave plaintexts, master aggregation keys, or enclave authentication keys.

C. Leakage-Aware Formal Security Model

At epoch e , the validated micro-batch is:

$$B_e = \{P_{e,1}, \dots, P_{e,n_e}\} \quad (126)$$

where each packet is generated during Phases II–III.

The outsourced encrypted object is:

$$\Omega_e = (BID_e, epoch_e, CT_e^{AES}, \mathcal{D}_e, CT_e^{L-ABE}, Root_e, \sigma_e) \quad (127)$$

where:

$$\mathcal{D}_e = \{(SubID_i, |C_i|, AAD_i, IV_i, Tag_i)\}_{i=1}^{r_e} \quad (128)$$

denotes the authenticated chunk manifest.

Over q epochs:

$$Hist_q = \{(M_e, A_e, \Omega_e)\}_{e=1}^q \quad (129)$$

We define the total leakage profile:

$$\mathcal{L} = (L_{\text{setup}}, L_{\text{enc}}, L_{\text{sched}}, L_{\text{ret}}) \quad (130)$$

1) *Setup Leakage*:

$$L_{\text{setup}}(1^\lambda) = (\lambda, |\Omega|, q, |PP|) \quad (131)$$

reveals only public system parameters and coarse deployment dimensions.

2) *Encryption and Outsourcing Leakage*:

$$\text{Verify}_{\text{Dilithium}}\left(pk_{FN}, \sigma, H(BID \parallel epoch_k \parallel CT_{AES} \parallel \mathcal{D}_k \parallel CT_{L-ABE} \parallel Root_k)\right) \quad (132)$$

This leakage exposes only:

- Batch size
- Chunk count
- Chunk lengths
- Policy dimensions
- Ciphertext dimensions
- Coarse timing information

No plaintexts, attribute semantics, aggregation states, session keys, enclave secrets, or master aggregation keys are revealed.

3) *Scheduler Leakage*:

$$L_{\text{sched}}(B_e) = (\Phi(B_e), j_e^*, r_e) \quad (133)$$

where $\Phi(B_e)$ denotes a coarse workload abstraction, j_e^* denotes the selected fog node, r_e denotes the enclave-parallel aggregation factor.

The scheduler may observe chunk parallelism through r_e , but cannot access plaintexts, chunk contents, aggregation states, or enclave-resident secrets.

4) *Retrieval Leakage*:

$$L_{\text{ret}}(e, u) = (e, \text{acc}(u, e), |M_e|) \quad (134)$$

where:

$$\text{acc}(u, e) \in \{0, 1\}$$

indicates policy satisfaction.

5) *Total Leakage*:

$$\mathcal{L}(Hist_q) = \left(L_{\text{setup}}, \{L_{\text{enc}}\}_{e=1}^q, \{L_{\text{sched}}\}_{e=1}^q, \{L_{\text{ret}}\}_{(e,u) \in Q_{\text{ret}}}\right) \quad (135)$$

D. Adaptive Leakage-Aware Indistinguishability

Definition 1 (Leakage-Aware Security Experiment).

Let Π denote Spider.

The experiment

$$\text{Expt}_{\Pi, \mathcal{A}}^{\text{LA-IND}}(\lambda)$$

proceeds as follows.

1) **Setup**:

$$(PP, MSK) \leftarrow \text{Setup}(1^\lambda)$$

and PP is given to $\mathcal{A}$

2) **Queries**:

- Key Query:

$$SK_u \leftarrow \text{KeyGen}(MSK, Attr_u)$$

- Outsource Query:

$$(M_e, A_e) \mapsto \Omega_e$$

- Retrieve Query: returns decryption or $\perp$
- Scheduler Query: returns only L_{sched}

3) **Challenge**: The adversary submits:

$$Hist_0^*, Hist_1^*$$

such that:

$$L_{\text{enc}}(Hist_0^*) = L_{\text{enc}}(Hist_1^*) \quad (136)$$

The challenger samples:

$$b \leftarrow \{0, 1\}$$

and returns challenge ciphertexts generated from $Hist_b^*$.

4) **Post-Challenge Queries**: The adversary continues oracle access under standard non-triviality restrictions.

5) **Guess**: The adversary outputs b'

Definition 2 (Leakage-Aware Security).

Spider is $\mathcal{L}$ -secure if for all QPT adversaries $\mathcal{A}$:

$$\text{Adv}_{\Pi, \mathcal{A}}^{\text{LA-IND}}(\lambda) = \left| \Pr[\text{Expt} = 1] - \frac{1}{2} \right| \leq \text{negl}(\lambda) \quad (137)$$

E. Bounded Scheduler Leakage

Lemma 1 (Bounded Scheduler Leakage).

The scheduler-visible information:

$$L_{\text{sched}}(B_e) = (\Phi(B_e), j_e^*, r_e)$$

reveals no plaintext contents, semantic attributes, aggregation states, session keys, chunk-encryption keys, or enclave-resident secrets beyond the defined leakage abstraction.

Proof. (1) Workload Abstraction

Let:

$$\Phi : \mathcal{B} \rightarrow \mathcal{C}$$

with:

$$|\mathcal{C}| \ll |\mathcal{B}|$$

Since:

$$|\{B \in \mathcal{B} : \Phi(B) = c\}| \gg 1$$

the mapping is many-to-one and non-invertible.

Thus, $\Phi(B_e)$ reveals only coarse workload properties.

(2) Fog Selection Leakage

The selected fog node:

$$j_e^* = \arg \min_j f(\Psi(F_j), \Phi(B_e))$$

depends only on workload abstraction and system state.

Thus:

$$j_e^* \perp (M_e, A_e) \mid L_{\text{sched}}$$

(3) Parallelism Leakage

The enclave parallelism factor:

$$r_e = g(\Phi(B_e), \Psi(E_{j,k}))$$

depends only on workload abstraction and enclave resources.

Although r_e exposes chunk-level parallelism, it reveals no aggregation plaintexts or secret-dependent information.

(4) Secret Independence

The scheduler accesses no enclave-resident secrets:

$$K_{AES}^{master}, K_{chunk}^{(i)}, K_{S,i}, m_i, \tau_i$$

Hence:

$$\Pr[\mathcal{A} \text{ learns } X \mid L_{\text{sched}}] = \Pr[\mathcal{A} \text{ learns } X]$$

for any secret variable X .

Therefore, scheduler-visible information is bounded by the defined leakage abstraction. $\square$

F. Main Leakage-Aware Security Theorem

Theorem 1 (Leakage-Aware Confidentiality of Spider).

Assume:

- 1) *Kyber* is IND-CCA secure
- 2) *KDF* is a quantum-secure PRF
- 3) *ChaCha20-Poly1305* and *AES-GCM* are IND-CPA and INT-CTXT secure
- 4) *Lattice CP-ABE* is selectively secure under MLWE
- 5) *Dilithium* is EUF-CMA secure
- 6) The hash function is collision resistant
- 7) Enclave MAC authentication is existentially unforgeable
- 8) TEEs provide confidentiality and integrity

Then *Spider* is $\mathcal{L}$ -secure

Proof. We prove security through a sequence of hybrid games.

G_0 (*Real Game*): The adversary receives real encrypted objects $\mathcal{L}_e$.

G_1 (*Randomize Session Keys*): Replace:

$$K_{S,i} = \text{KDF}(H(k_{\text{kem},i}) \parallel R_{\text{secret},i} \parallel ID_i)$$

with uniformly random strings.

Then:

$$|\Pr[G_0] - \Pr[G_1]| \leq \text{Adv}_{KDF}^{\text{PRF}}(\lambda)$$

G_2 (*Randomize Kyber Secrets*): Replace Kyber shared secrets with random strings.

Then:

$$|\Pr[G_1] - \Pr[G_2]| \leq \text{Adv}_{Kyber}^{\text{IND-CCA}}(\lambda)$$

G_3 (*Randomize Device AEAD*): Replace ChaCha20-Poly1305 ciphertexts with encryptions of zero plaintexts.

Then:

$$|\Pr[G_2] - \Pr[G_3]| \leq \text{Adv}_{\text{ChaCha20-Poly1305}}^{\text{AEAD}}$$

G_4 (*Randomize Aggregation Chunks*): Replace all enclave-generated aggregation chunks:

$$\{(C_i, \text{Tag}_i)\}_{i=1}^{r_e}$$

with encryptions of zero chunks under simulated keys.

Then:

$$|\Pr[G_3] - \Pr[G_4]| \leq \text{Adv}_{\text{AES-GCM}}^{\text{AEAD}}$$

$G_{4.5}$ (*Simulate Aggregation Commitment*): Replace:

$$\text{Root}_e = H(\text{BID}_e \parallel \text{epoch}_e \parallel r_e \parallel h_{(1)} \parallel \dots \parallel h_{(r_e)})$$

with uniformly random strings.

Under collision resistance and enclave MAC unforgeability:

$$|\Pr[G_4] - \Pr[G_{4.5}]| \leq \text{negl}(\lambda)$$

G_5 (*Simulate CP-ABE Header*): Replace:

$$CT_e^{L-ABE}$$

with simulated values generated using only:

$$(|CT|, \ell(A_e), \tau(A_e))$$

Then:

$$|\Pr[G_{4.5}] - \Pr[G_5]| \leq \text{Adv}_{\text{L-CP-ABE}}^{\text{SEL-CPA}}$$

G_6 (*Simulate Signatures*): Replace:

$$\sigma_e$$

with uniformly random strings.

Then:

$$|\Pr[G_5] - \Pr[G_6]| \leq \text{Adv}_{\text{Dilithium}}^{\text{EUF-CMA}}$$

G_7 (*Ideal Game*): All outputs are generated solely from leakage $\mathcal{L}$.

Thus:

$$\begin{aligned} \text{Adv}_{\Pi, \mathcal{A}}^{\text{LA-IND}} &\leq \text{Adv}_{KDF}^{\text{PRF}} + \text{Adv}_{Kyber}^{\text{IND-CCA}} \\ &\quad + \text{Adv}_{\text{ChaCha20-Poly1305}}^{\text{AEAD}} + \text{Adv}_{\text{AES-GCM}}^{\text{AEAD}} \\ &\quad + \text{Adv}_{\text{L-CP-ABE}}^{\text{SEL-CPA}} + \text{Adv}_{\text{Dilithium}}^{\text{EUF-CMA}} \\ &\quad + \text{negl}(\lambda) \end{aligned} \quad (138)$$

Since all terms are negligible, Spider is $\mathcal{L}$ -secure. $\square$

G. Integrity and Authenticity

Theorem 2 (End-to-End Integrity and Authenticity).

Assume:

- Device MAC authentication is secure
- ChaCha20-Poly1305 and AES-GCM satisfy INT-CTXT security
- Enclave MAC authentication is unforgeable
- Dilithium is EUF-CMA secure
- The hash function is collision resistant

Then any QPT adversary that forges valid packets, modifies aggregation chunks undetected, manipulates chunk manifests, reorders chunks, duplicates chunks, omits sub-batches, or produces forged fog outputs succeeds only with negligible probability.

Proof. Spider provides four-layer integrity protection.

(1) Device Authenticity

Each packet includes:

$$Auth_i = MAC_{K_{S,i}}(ID_i \parallel Metadata_i \parallel CT_i \parallel Tag_i)$$

Therefore, successful forgery breaks MAC unforgeability.

(2) Chunk Integrity

Each enclave-generated chunk:

$$(C_i, Tag_i)$$

is protected by AES-GCM authentication.

Any successful ciphertext modification without detection breaks INT-CTXT security.

(3) Enclave Output Authenticity

Each enclave generates:

$$\tau_i = MAC_{K_i^{TEE}}(h_i)$$

Thus:

- Forged chunk metadata
- Fake enclave outputs
- Modified chunk descriptors

require breaking enclave MAC unforgeability.

(4) Aggregation Integrity

Spider computes:

$$Root_e = H(BID_e \parallel epoch_e \parallel r_e \parallel h_{(1)} \parallel \dots \parallel h_{(r_e)})$$

This commitment binds:

- Deterministic chunk ordering
- Chunk completeness
- Replay resistance
- Duplication resistance
- Omission resistance

Any replay, duplication, omission, or reordering attack changes:

$$Root_e$$

and causes verification failure.

(5) Fog Authenticity

Users verify:

$$Verify_{pk_{FN}}(\sigma_e, H(BID_e \parallel epoch_e \parallel CT_e^{AES} \parallel \mathcal{D}_e \parallel CT_e^{L-ABE} \parallel Root_e))$$

A valid forgery implies breaking Dilithium EUF-CMA security.

By union bound:

$$\begin{aligned} Adv_{\Pi, A}^{\text{forge}} &\leq Adv^{\text{MAC}} + Adv_{\text{ChaCha20-Poly1305}}^{\text{INT-CTXT}} \\ &\quad + Adv_{\text{AES-GCM}}^{\text{INT-CTXT}} + Adv_{\text{TEE}}^{\text{MAC}} \\ &\quad + Adv_{\text{Dilithium}}^{\text{EUF-CMA}} + \text{negl}(\lambda) \end{aligned} \quad (139)$$

Thus Spider achieves end-to-end integrity and authenticity. $\square$

H. Deterministic Aggregation Correctness

Theorem 3 (Deterministic Aggregation Correctness).

If:

- All enclave outputs satisfy MAC verification,
- Chunk manifests are valid
- Deterministic ordering $\mathcal{O}$ is preserved
- $Root'_e = Root_e$

then the reconstructed aggregation:

$$M_{agg} = \mathcal{M}_{i \in \mathcal{O}} \mathcal{M}_i$$

contains every valid sub-batch exactly once and is identical to the enclave-generated aggregation.

Proof. By construction:

$$\bigcup_i B_e^{(i)} = B_e$$

and:

$$B_e^{(i)} \cap B_e^{(j)} = \emptyset$$

Thus every packet appears in exactly one enclave sub-batch. Deterministic ordering:

$$\mathcal{O} = \text{SortBy}(\text{SubID}_i)$$

ensures unique reconstruction order.

Since:

$$Root'_e = Root_e$$

all chunk digests:

$$h_{(1)}, \dots, h_{(r_e)}$$

match the original enclave-generated outputs.

Therefore:

- No replay occurred
- No chunk omission occurred
- No duplication occurred
- No reordering occurred

Hence the reconstructed aggregation equals the original enclave-generated aggregation. $\square$

I. Policy Privacy

Theorem 4 (Policy Privacy up to Structural Leakage).

Assuming hash collision resistance and selective security of lattice CP-ABE, the adversary learns only policy dimensions and structural form, but not attribute semantics.

Proof. Attributes are encoded as:

$$ID_j = H(attr_j)$$

which hides semantic labels.

The ciphertext reveals only:

$$(\ell(A_e), \tau(A_e))$$

Under selective security of lattice CP-ABE, no additional attribute information is leaked beyond the permitted structural leakage. $\square$

J. Forward Secrecy of Session and Chunk Protection

Device-to-fog session keys are derived as:

$$K_{S,i} = KDF(H(k_{kem,i}) \parallel R_{secret,i} \parallel ID_i \parallel t_i \parallel ctr_i) \quad (140)$$

Aggregation chunk keys are derived as:

$$K_{chunk}^{(i)} = KDF(K_{AES}^{master} \parallel BID \parallel SubID_i \parallel epoch_k) \quad (141)$$

Assuming KDF pseudorandomness:

- Compromise of one chunk key does not reveal other chunk keys
- Compromise of later epochs does not reveal prior epochs
- Compromise of one aggregation chunk does not reveal future aggregation keys

Therefore, Spider preserves forward secrecy for both device-level session protection and enclave-generated aggregation encryption.

Theorem 5 (Forward Secrecy of Session Keys).

Assume Kyber is IND-CCA secure and KDF is pseudorandom. Then compromise of later system states does not reveal any past session key except with negligible probability.

Proof. Let i^* be a completed session. The adversary $\mathcal{A}$ obtains all transcripts, including c_{kem,i^*} , and arbitrary later system states, but not the decapsulated secret k_{kem,i^*} inside the TEE.

In the indistinguishability experiment, the challenger returns

$$K^* = \begin{cases} K_{S,i^*}, & b = 0, \\ U_\kappa, & b = 1, \end{cases} \quad b \leftarrow \{0, 1\}$$

Game G_0 : Real key

$$K_{S,i^*} = KDF(H(k_{kem,i^*}) \parallel \dots)$$

Game G_1 : Replace k_{kem,i^*} with $\tilde{k} \leftarrow \{0, 1\}^\ell$:

$$|\Pr[G_0] - \Pr[G_1]| \leq Adv_{Kyber}^{IND-CCA}$$

Game G_2 : Replace KDF output with U_κ :

$$|\Pr[G_1] - \Pr[G_2]| \leq Adv_{KDF}^{PRF}$$

In G_2 , K^* is uniform, so $\Pr[G_2 = 1] = \frac{1}{2}$

Thus,

$$Adv_{\Pi, \mathcal{A}}^{fs} \leq Adv_{Kyber}^{IND-CCA} + Adv_{KDF}^{PRF} \leq \text{negl}(\lambda)$$

Fresh $(c_{kem,i}, t_i, ctr_i)$ ensures independence across sessions, completing the proof. $\square$

VII. EVALUATION

This section presents the evaluation of the scheme by illustrating the functionality comparison and demonstrating the experiments conducted for measuring the processing time of encryption, decryption and load balancing.

A. Computation Cost Analysis

To provide a fair and structured evaluation, we divide the computation cost analysis into two groups: (i) post-quantum cryptographic protection schemes and (ii) fog load-balancing mechanisms. This separation is necessary because existing works typically focus on either cryptographic protection or scheduling efficiency, whereas the proposed framework integrates both aspects.

Experiment Setup: The experiments were conducted on a host system powered by an Intel(R) Core(TM) i5-10600 CPU @ 3.30 GHz with 11 GiB of allocated system memory. Within this environment, fog nodes were implemented using VirtualBox instances to emulate a distributed setting. To further simulate the Trusted Execution Environment (TEE) inside each node, QEMU was employed, enabling secure enclave operations along with memory isolation for protected execution.

To make the evaluation setting clear, Table III summarizes the number of tasks, fog nodes, and TEE enclaves used in each graph. The same fog-node configuration is not used for all experiments because each graph evaluates a different component of the proposed framework. Graphs 1, 3, and 4 focus on cryptographic latency and therefore use a single-node or user-side execution setting rather than a distributed fog cluster. Graph 2 evaluates cache/reuse-aware scheduling by varying the number of tasks while fixing the number of fog nodes. Graphs 5 and 6 vary the number of fog nodes to evaluate inter-node load-balancing scalability under homogeneous and heterogeneous fog conditions. Graph 7 uses a larger fog cluster to evaluate recovery latency under node failure. Graphs 8–9 focus on intra-node scheduling inside one fog node with four TEE enclaves.

1) Post-Quantum Protection Cost Comparison. We compare our scheme with representative post-quantum protection works [4], [35], and [36], focusing on key generation, device/edge encryption, fog-side processing, and user decryption. Table IV summarizes the notations used in the computation cost analysis.

TABLE II
COMPUTATION COST COMPARISON

Scheme	KeyGen	Encrypt (Edge)	Encrypt (Fog)	Decrypt
[4]	$(N_{\text{strict}} + N_{\text{flexible}} + 1)H$ + $N_{\text{flexible}} \text{PRF}$ + $CD_{\text{keygen}} + \text{EpochInit}$	AES-GCM + $(N_a + 3)SG$ + $(N_a + 2)Mul_R$ + $(N_a + 3)Add_b + N_a H$ + MAC_{gen}	$MAC_{\text{ver}} + F Mul_R$ + $Add_R + Sign$	<i>Device:</i> $MAC_{\text{ver}} + N_{\text{strict_used}} Mul_R$ + $(2N_{\text{strict_used}} + 2)Add_R$ + $H + T_{\text{decode}}$ + AES-GCM <i>Fog:</i> $Ver + N_{\text{flex_used}} Mul_R$ + $Add_R + MAC_{\text{gen}}$
[35]	$O(T_{\text{KyberKG}} + T_{\text{Dil.KG}})$	$O(T_{\text{KyberEnc}} + T_{\text{Dil.Sign}})$	–	$O(T_{\text{KyberDec}})$
[36]	–	$O(N^2)$ (Chaotic-map Based)	–	$O(N^2)$
Ours	$O(T_{\text{KyberKG}} + T_{\text{L-ABE.Setup}})$	$O(T_{\text{PUF}} + T_{\text{KyberEnc}} + T_{\text{ChaChaEnc}} + T_{\text{MAC}})$	$O\left(\frac{ B_k }{\sum_j m_j} \cdot (T_{\text{KyberDec}} + T_{\text{ChaChaDec}})\right)$ + $O(T_{\text{AESEnc}})$ + $O\left(\frac{ S }{\sum_j m_j} \cdot T_{\text{L-ABE.Enc}}\right)$ + $O(T_{\text{Dil.Sign}} + T_{\text{sched}})$	$O(T_{\text{L-ABE.Dec}} + T_{\text{AESDec}})$

TABLE III
EXPERIMENTAL CONFIGURATION USED IN EACH EVALUATION GRAPH

Graph	Evaluation Focus	Tasks	Fog Nodes	TEE Enclaves
Graph 1	CP-ABE setup latency	–	1	–
Graph 2	Cache reuse at edge gateway	100-2000	5	–
Graph 3	CP-ABE encryption latency at fog node	20	1	–
Graph 4	User decryption latency	20	user-side process	–
Graph 5	Homogeneous load-balancing latency	500	2, 4, 6, 8, 10, 12	–
Graph 6	Heterogeneous fog-node load balancing	500	2, 4, 6, 8, 10, 12	–
Graph 7	Recovery latency under node failure	500	20	–
Graphs 8–9	Intra-node scheduling and queue diagnosis	500	1	4

Table II presents a comparative analysis of the proposed scheme against representative approaches, highlighting differences in computational complexity, system design, and scalability.

Scheme [4] incurs relatively high overhead due to its reliance on lattice-based CP-ABE with fine-grained access policies. The key generation cost scales with the number of attributes, while encryption and decryption involve multiple polynomial operations, Gaussian sampling, and modular arithmetic. As a result, its computational cost is strongly attribute-dependent, leading to increased latency, especially in large-scale deployments. Moreover, authentication and verification operations are embedded across device and fog layers, further contributing to the overall processing burden.

Scheme [35] adopts standalone post-quantum cryptographic primitives, where the computational cost is dominated by Kyber encapsulation/decapsulation and Dilithium signature generation. Each operation incurs a constant-time complexity, i.e., $O(1)$ per invocation. However, the scheme lacks integrated access control and distributed processing capabilities, limiting its scalability and making it less suitable for large-scale IIoT environments with high data volume and concurrency.

Scheme [36] follows a transformation-based design using chaotic-map encoding mechanisms. Its computational cost is dominated by iterative transformation and mapping processes, resulting in an overall complexity of $O(N^2)$ for encryption

and decryption. Unlike structured or parallelizable designs, this quadratic dependence on data size significantly limits scalability as the dataset grows. Furthermore, the absence of distributed processing or workload partitioning makes it less adaptable to dynamic IIoT environments.

In contrast, the proposed scheme achieves a scalable and efficient computation model by combining lightweight device-side operations with fog-assisted parallel processing. At the device layer, the cost is dominated by efficient primitives such as Kyber encapsulation, ChaCha20-Poly1305 encryption, and PUF-based authentication, ensuring low overhead suitable for resource-constrained IIoT devices. At the fog layer, computationally intensive tasks, including batch decryption, CP-ABE encryption, and signature generation, are offloaded and parallelized across multiple enclaves.

As shown in Table II, the fog-side complexity is effectively reduced by a factor proportional to $\frac{|B_k|}{\sum_j m_j}$ for batch processing and $\frac{|S|}{\sum_j m_j}$ for attribute-based encryption, where $|B_k|$ denotes the batch size and $\sum_j m_j$ represents the total number of available enclaves. This parallel execution model significantly lowers processing latency and enables horizontal scalability.

Overall, unlike schemes with attribute-dependent overhead [4], limited system integration [35], or quadratic data-dependent complexity [36], the proposed framework provides a unified, distributed, and scalable computation model that is

TABLE IV
NOTATION FOR COMPUTATION COST ANALYSIS

Symbol	Description
<i>System Parameters</i>	
$ F $	Number of fog nodes
m_j	Number of enclaves in fog node F_j
$ S $	Number of attributes in access policy
N	Input data size
<i>Lattice / CP-ABE Operations</i>	
T_{poly}	Polynomial (NTT-based) operation cost
T_{PRF}	Pseudorandom function cost
$T_{\text{L-ABE.Setup}}$	CP-ABE setup cost
$T_{\text{L-ABE.Enc}}$	CP-ABE encryption cost
$T_{\text{L-ABE.Dec}}$	CP-ABE decryption cost
<i>Post-Quantum Primitives</i>	
T_{Kyber}	Kyber encapsulation/decapsulation cost
T_{Dil}	Dilithium signing/verification cost
<i>Symmetric Operations</i>	
T_{AE}	AEAD encryption/decryption cost
T_{MAC}	MAC generation/verification cost
T_{PUF}	PUF evaluation cost
<i>Scheduling (Ours)</i>	
T_{sched}	Scheduling overhead
$\sum_j m_j$	Total available enclaves
$\frac{ S }{\sum_j m_j}$	Parallel workload factor

TABLE V
NOTATION FOR LOAD BALANCING COST ANALYSIS

Symbol	Description
$ F $	Number of fog nodes
m_j	Number of enclaves in fog node F_j
T	Number of incoming tasks
N_{usage}	Total network traffic volume
$ MC $	Number of edge/medical centers
P	Number of workload requests/tasks
R	Number of regions
I	Number of learning iterations
d	Average neighbor degree

well-suited for large-scale and latency-sensitive IIoT environments.

2) Load Balancing Cost Analysis We analyze the computational and communication overhead of load balancing mechanisms across representative fog/edge scheduling schemes. Table V summarizes the notations used in this analysis.

Table VI compares the LB strategies and associated costs of representative schemes. Since the compared works adopt different architectures, we report dominant scheduling and communication costs derived from their algorithms.

Existing schemes generally rely on single-level or iterative scheduling mechanisms with varying computational overhead. Scheme [22] assigns each incoming task by scanning all candidate fog nodes and selecting the minimum-latency node, leading to a scheduling cost of $O(T|F|)$. Its communication overhead depends on total network traffic, modeled as $O(N_{\text{usage}})$, which grows with system workload and inter-layer data exchange.

Scheme [37] improves adaptability by combining static

planning and SDN-assisted dynamic control. However, its scheduling cost is dominated by pairwise coordination and workload distribution across centers, resulting in $O(|MC|^2 + |MC| + P)$. The communication overhead scales with task distribution and inter-node coordination, captured as $O(Pd)$.

Scheme [39] adopts a distributed reinforcement learning-based scheduling mechanism. Its dominant cost arises from iterative Q-learning across regions, fog nodes, and tasks, yielding a complexity of $O(RI|F|T)$. Additionally, communication overhead is introduced through Q-value sharing among neighboring regions, modeled as $O(RId)$.

In contrast, the proposed scheme introduces a hierarchical and adaptive LB mechanism that jointly considers fog nodes and enclave-level resources. The scheduling complexity is reduced to $O(|F| + \sum_j m_j)$, where $\sum_j m_j$ denotes the total number of enclaves across fog nodes. The communication overhead follows the same bound, as only lightweight state information and scheduling decisions are exchanged. This two-level design enables fine-grained parallelism across enclaves, significantly improving scalability and reducing bottlenecks compared to single-level or learning-based approaches.

B. Performance Analysis

This section evaluates the performance of the proposed scheme in terms of computational overhead and load-balancing efficiency.

1) Cryptographic and Scheduling Overhead Comparison: To analyze the cryptographic and scheduling-related overhead of the proposed framework, we measure the main processing stages in the secure fog pipeline. The comparison is illustrated through the following four metrics:

- 1) CP-ABE Setup Latency
- 2) Cache/Reuse-aware Scheduling Latency
- 3) CP-ABE Encryption Cost at Fog Layer
- 4) CP-ABE Decryption Cost at User side

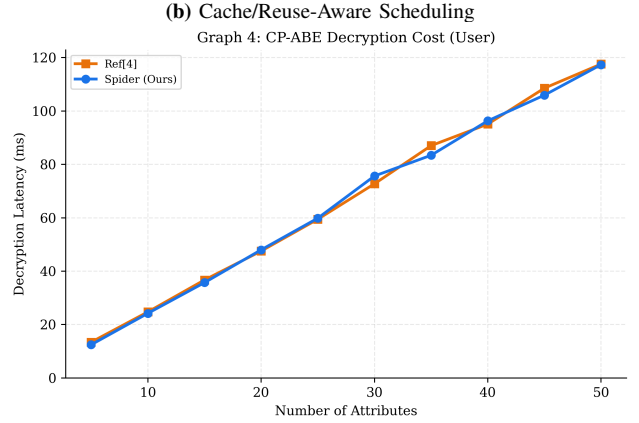
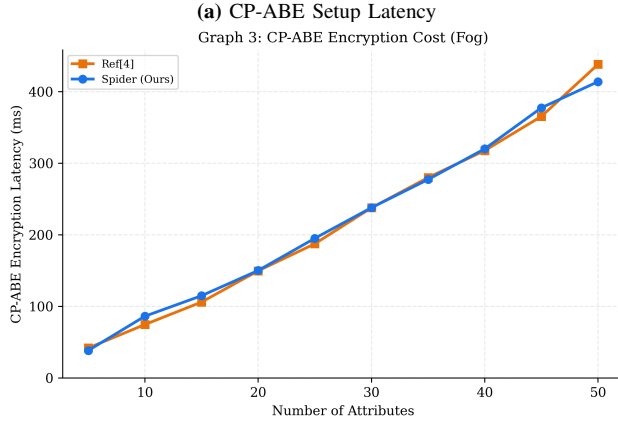
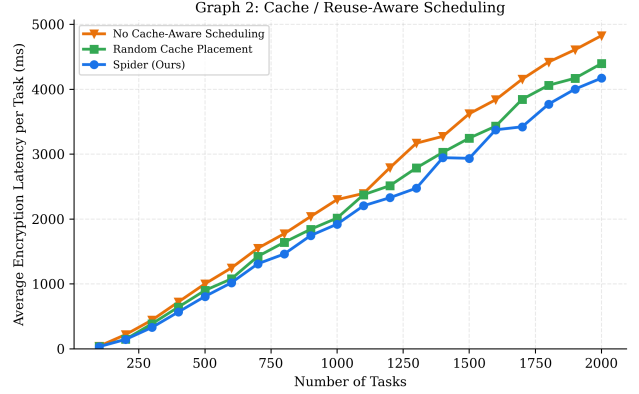
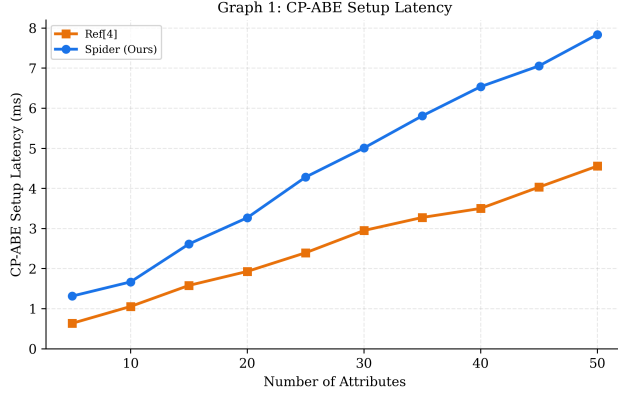
Fig. 2(a) shows that CP-ABE Setup Latency increases with the number of attributes for both schemes. Spider has higher setup latency than [4] because it prepares additional reusable policy and cryptographic structures for later fog-side processing. Since this cost is incurred only during initialization, the overhead is acceptable as a one-time preparation cost.

Fig. 2(b) shows that scheduling latency increases as the number of tasks grows. The no-cache baseline has the highest latency, while random cache placement provides limited improvement. Spider achieves the lowest latency because it considers reusable policy structures and cryptographic components during scheduling, reducing redundant fog-side computation.

Fig. 2(c) compares CP-ABE Encryption Cost at the fog layer. The latency increases almost linearly with the number of attributes because larger policies require more CP-ABE components and polynomial operations. Spider shows slightly higher or comparable latency compared with [4]. This is expected because Spider adds TEE-assisted split-phase processing, cache/reuse-aware management, and Spider-based fog orchestration on top of fog-side CP-ABE encryption. However,

TABLE VI
LOAD BALANCING COST COMPARISON

Scheme	LB Strategy	Scheduling Cost	Communication Cost	Scalability / Scope
[22]	Optimised Load Balancing (OLB)	$O(T F)$	$O(N_{usage})$	Single-level Fog Scheduling
[37]	Adaptive SDN-based Load Balancing	$O(MC ^2 + MC + P)$	$O(Pd)$	Multi-tier Edge/Fog with SDN Controller
[39]	Distributed Reinforcement Learning-based Task Scheduling	$O(RI F T)$	$O(RId)$	Multi-round Distributed Learning-based Scheduling
Ours	Hierarchical Adaptive Fog and Enclave Scheduling	$O(F + \sum_j m_j)$	$O(F + \sum_j m_j)$	Two-level Scheduling Fog Layer and Enclave



(c) CP-ABE Encryption Cost at Fog

(d) CP-ABE Decryption Cost at User

Fig. 2. Computation-cost and Cache-Aware Scheduling performance of the proposed Spider framework

the gap remains small, indicating only moderate fog-side overhead.

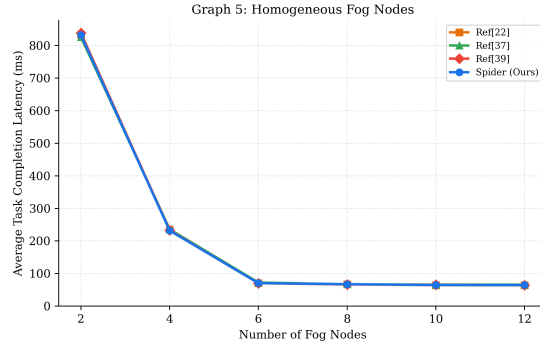
Fig. 2(d) shows the CP-ABE Decryption Cost at User side. The latency increases with the number of attributes because more ciphertext components must be evaluated during key recovery. Spider achieves slightly lower user-side decryption latency than [4], showing that the additional fog-layer mechanisms do not transfer heavy computation to the authorized user.

Overall, Spider introduces moderate setup and fog-side encryption overhead, but achieves lower cache/reuse-aware scheduling latency and slightly lower user-side decryption cost. Although [4] is already a lightweight quantum-resistant CP-ABE access-control scheme, Spider provides broader IIoT

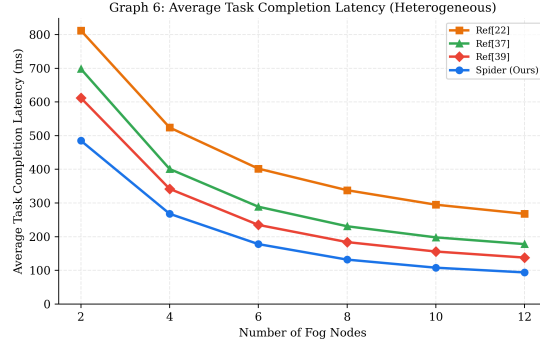
and fog-layer functions, including PUF authentication, TEE-assisted processing, cache/reuse-aware scheduling, and hierarchical load balancing, without introducing prohibitive cryptographic overhead.

2) *Load-Balancing Performance Analysis*: To evaluate the effectiveness of the proposed load-balancing mechanism, we measure both inter-node and intra-node scheduling behavior in the secure fog pipeline. The inter-node evaluation compares Spider with representative fog/edge load-balancing schemes from [22], [37] and [39] while the intra-node evaluation uses OP-TEE QEMU to emulate heterogeneous TEE secure-world execution contexts. The comparison is illustrated through the following five metrics:

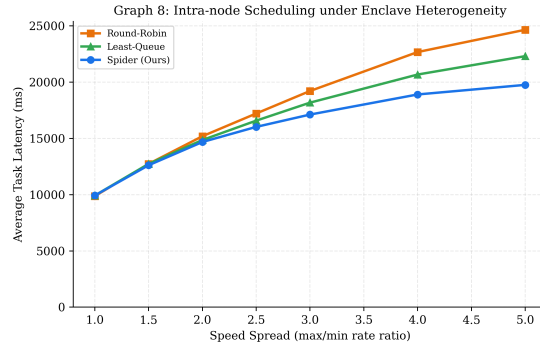
- 1) Homogeneous fog-node latency



(a) Homogeneous fog-node latency



(b) Heterogeneous fog-node latency



(c) Intra-node scheduling under OP-TEE heterogeneity

Fig. 3. Load-balancing and recovery performance of the proposed Spider framework under homogeneous, heterogeneous, failure-prone, and OP-TEE-based intra-node scheduling environments

2) Heterogeneous fog-node latency

3) Intra-node scheduling under OP-TEE heterogeneity

Fig. 3(a) shows the average task-completion latency under a homogeneous fog-node environment. Since all fog nodes have similar processing capacity and network delay, the compared schemes achieve similar latency once the number of fog nodes becomes sufficient. Spider remains comparable to the baselines, showing that its additional scheduling logic does not introduce noticeable overhead when the fog environment is uniform.

Fig. 3(b) shows the heterogeneous fog-node scenario, where fog nodes differ in processing capacity, network delay, and runtime condition. In this case, Spider achieves lower average task-completion latency than the baselines. This improvement comes from its capability-aware scheduling model, which

considers node capacity, queue state, network latency, and secure-processing readiness before assigning tasks.

Fig. 3(c) evaluates intra-node scheduling under OP-TEE heterogeneity. In this experiment, OP-TEE QEMU is used to emulate secure-world execution contexts with different service rates. When the speed spread is small, all methods perform similarly because the secure execution contexts have nearly equal capability. As the speed spread increases, Spider achieves lower latency because it considers both queue state and secure-world service rate, while Round-Robin and Least-Queue do not fully capture TEE execution heterogeneity.

Overall, Spider is most beneficial in heterogeneous, TEE-constrained, and failure-prone fog environments. In homogeneous settings, it remains comparable to existing fog-level scheduling baselines, indicating low scheduling overhead. In

heterogeneous settings, it provides clearer benefits by jointly considering fog-node capability, queue state, network latency, secure-world service rate, cache affinity, and failure recovery.

C. Fault-Tolerance and Secure Recovery Evaluation

To evaluate the improved fault-tolerance mechanism of Spider, we measure recovery behavior under fog-node failures. This experiment focuses on the effectiveness of group-based failure detection, quorum-based confirmation, secure delegation capsules, and capability-aware recovery-node selection. This complements the existing recovery-time evaluation by isolating the benefit of the newly added decentralized fault-tolerant recovery mechanism.

1) *Experimental Setup*: We emulate a heterogeneous fog environment with $N = \{10, 20, 30, 40, 50\}$ fog nodes. Each fog node hosts multiple TEE enclaves with different service rates and EPC availability. Validated micro-batches are decomposed into enclave-parallel sub-batches, following the Spider processing model. During execution, a percentage of fog nodes is randomly failed at runtime, with failure rates varied as:

$$p_f \in \{5\%, 10\%, 15\%, 20\%, 25\%\}$$

A failure is injected by stopping the selected fog node and preventing it from sending heartbeat messages. Spider detects failures using authenticated group-based heartbeat monitoring and declares a node failed only after quorum confirmation. Unfinished sub-batches are then reassigned using the existing SpiderScore-based recovery selection.

2) *Compared Fault-Tolerance Baselines*: We compare Spider-FT with the following baselines:

- **No Fault-Tolerance**: Failed workloads are dropped or restarted manually.
- **Centralized Heartbeat**: All fog nodes report heartbeat messages directly to the MFN. Failed workloads are reassigned after timeout detection.
- **Full Checkpoint Replication**: Each workload periodically replicates full execution state to a backup node. Recovery resumes from the latest checkpoint but incurs high synchronization overhead.
- **Round-Robin Recovery**: Failed workloads are reassigned to the next available fog node without considering capability, enclave state, or EPC pressure.
- **Least-Queue Recovery**: Failed workloads are reassigned to the node with the shortest queue, without considering secure-execution readiness.
- **Spider Fault-Tolerance**: The proposed mechanism using group-based heartbeat detection, quorum confirmation, secure delegation capsules, and SpiderScore-based recovery.

3) *Evaluation Metrics*: We use the following metrics:

- **Failure Detection Time**: Time between node failure injection and confirmed failure declaration.
- **Recovery Latency**: Time required to resume unfinished workloads on a recovery fog node.
- **Task Completion Ratio**: Percentage of workloads successfully completed after failures.

- **Recomputation Overhead**: Amount of repeated processing caused by failure recovery.
- **Control-Plane Overhead**: Number of heartbeat and recovery-control messages exchanged.
- **False-Positive Rate**: Percentage of healthy nodes incorrectly declared failed.

4) *Expected Results and Analysis*: The expected result is that **Spider Fault-Tolerance** achieves lower recovery latency than centralized heartbeat, round-robin recovery, and least-queue recovery because unfinished workloads are recovered at the sub-batch level rather than restarting the whole batch. This directly follows Spider's enclave-parallel decomposition, where completed sub-batches remain valid and only incomplete fragments are reassigned.

Compared with **Full Checkpoint Replication**, Spider-FT is expected to introduce significantly lower control-plane overhead because it avoids replicating full enclave memory. Instead, it preserves only lightweight recovery metadata and partially processed cryptographic state through the secure delegation capsule:

$$\Xi_k = (ID_k, SubID_k, Prog_k, Meta_k, CT_k^{partial}, T_k, epoch_k, \sigma_k)$$

Compared with **Centralized Heartbeat**, Spider-FT should reduce monitoring overhead because heartbeat exchange is localized within bounded-size monitoring groups:

$$G_i = \{F_1, F_2, \dots, F_s\}, \quad 3 \leq s \leq 7$$

Thus, the per-node monitoring cost remains bounded by $O(s)$ rather than depending on the total number of fog nodes. Quorum-based confirmation further reduces false positives under transient network delay.

Overall, Spider-FT is expected to provide the best trade-off among recovery latency, task completion ratio, recomputation overhead, and monitoring cost. The improvement comes from four combined factors: decentralized heartbeat monitoring, quorum-based failure confirmation, lightweight secure workload preservation, and capability-aware recovery-node selection.

VIII. CONCLUSION

This paper proposed Spider, a post-quantum secure and dynamic load-balanced encryption framework for IIoT data in fog computing environments. The proposed framework integrates PUF-based device authentication, Kyber-based post-quantum key establishment, ChaCha20-Poly1305 authenticated encryption, TEE-assisted secure fog processing, and lattice-based CP-ABE for fine-grained access control. To support scalable fog-layer processing, Spider introduces a hierarchical scheduling model that performs inter-node load balancing across fog nodes and intra-node scheduling across secure execution contexts.

The evaluation results show that Spider introduces only moderate setup and fog-side encryption overhead compared with a lightweight quantum-resistant CP-ABE baseline. This

overhead is mainly caused by the additional reusable policy structures, TEE-assisted processing, and fog orchestration mechanisms. However, Spider reduces cache/reuse-aware scheduling latency and maintains slightly lower user-side decryption cost, showing that the additional fog-layer mechanisms do not shift heavy computation to authorized users.

The load-balancing results further demonstrate that Spider is most beneficial in heterogeneous, TEE-constrained, and failure-prone fog environments. In homogeneous settings, its performance remains comparable to existing fog-level scheduling schemes, indicating that the additional scheduling logic does not introduce significant overhead. In heterogeneous settings, Spider achieves lower task-completion latency by considering fog-node capability, queue state, network delay, secure-world service rate, cache affinity, and failure-aware delegation. The OP-TEE QEMU-based intra-node evaluation also shows that Spider can exploit secure execution heterogeneity by routing more tasks to faster secure-world contexts instead of treating all contexts equally.

Overall, Spider provides a practical trade-off between post-quantum security, fine-grained access control, TEE-assisted secure processing, and adaptive fog-layer workload orchestration. The proposed framework is therefore suitable for large-scale IIoT environments where secure data sharing, low-latency processing, and resilience under heterogeneous fog conditions are required. Future work will extend the prototype to real hardware-based TEE platforms and evaluate the framework under larger IIoT workloads, stronger failure models, and realistic industrial network traces.

REFERENCES

- [1] S. Fugkeaw, A. Changtor, T. Maneerat, P. Rattanasrisuk, and K. Tangtanawirut, "LightPUF-IIoT: A Lightweight PUF-Based Authentication Scheme With Real-Time Detection of Rogue Devices in Fog-Assisted IIoT Data Sharing," *IEEE Open Journal of the Computer Society*, vol. 6, pp. 1438–1450, 2025, doi: 10.1109/OJCS.2025.3607984.
- [2] A. Changtor, T. Maneerat, P. Rattanasrisuk, K. Tangtanawirut, and S. Fugkeaw, "A Secure and Lightweight IIoT Data Authentication Using PUF-Based in IoT Fog-Assisted Cloud," in *Proc. 2025 17th International Conference on Knowledge and Smart Technology (KST)*, 2025, pp. 40–45, doi: 10.1109/KST65016.2025.11003299.
- [3] Y. Yang, J. Zhang, X. Liu, and J. Ma, "A Scalable and Auditable Secure Data Sharing Scheme With Traceability for Fog-Based Smart Logistics," *IEEE Internet of Things Journal*, vol. 10, no. 10, pp. 8603–8617, 2023, doi: 10.1109/IIOT.2022.3220850.
- [4] P. Poomekum, A. Suriyawong, and S. Fugkeaw, "Fine-Grained and Lightweight Quantum-Resistant Access Control System With Efficient Revocation for IoT Cloud," *IEEE Open Journal of the Communications Society*, vol. 6, pp. 8652–8666, 2025, doi: 10.1109/OJCOMS.2025.3620094.
- [5] R. P. Gupta, C. Vorakulpipat, S. Fugkeaw, and A. Witayangkurn, "A Scalable and Secure Outsourced IoT Electronic Health Records with Efficient User Revocation Using Fog-Assisted Cloud Model," in *Proc. 2024 21st International Joint Conference on Computer Science and Software Engineering (JCSSE)*, 2024, pp. 61–67, doi: 10.1109/JCSSE61278.2024.10613723.
- [6] Z. Wang and Y. Zhou, "Analysis and Evaluation of Intel Software Guard Extension-Based Trusted Execution Environment Usage in Edge Intelligence and Internet of Things Scenarios," *Future Internet*, vol. 17, no. 1, Art. no. 32, 2025, doi: 10.3390/fi17010032.
- [7] A. Muñoz, R. Ríos, R. Román, and J. López, "A Survey on the (In)Security of Trusted Execution Environments," *Computers & Security*, vol. 129, Art. no. 103180, 2023, doi: 10.1016/j.cose.2023.103180.
- [8] A. N. Alvi, B. Ali, M. S. Saleh, M. Alkhatami, D. Alsadie, and B. Alghamdi, "Secure Computing for Fog-Enabled Industrial IoT," *Sensors*, vol. 24, no. 7, Art. no. 2098, 2024, doi: 10.3390/s24072098.
- [9] N. Sarwar, R. S. Alharthi, H. M. Mujlid, et al., "Enhancing IoT Communication in Fog Computing: A Lightweight Remote User Authentication and Key Management Scheme Utilizing ECC," *Journal of Cloud Computing*, vol. 14, Art. no. 86, 2025, doi: 10.1186/s13677-025-00816-y.
- [10] W. B. Daoud, S. Othmen, M. Hamdi, et al., "Fog Computing Network Security Based on Resources Management," *Journal of Wireless Communications and Networking*, vol. 2023, Art. no. 50, 2023, doi: 10.1186/s13638-023-02256-1.
- [11] H. Nguyen, S. Huda, Y. Nogami, and T. T. Nguyen, "Security in Post-Quantum Era: A Comprehensive Survey on Lattice-Based Algorithms," *IEEE Access*, vol. 13, pp. 89003–89024, 2025, doi: 10.1109/ACCESS.2025.3571307.
- [12] A. B. Shahid, K. Mansoor, Y. A. Bangash, et al., "Post-Quantum Cryptographic Authentication Protocol for Industrial IoT Using Lattice-Based Cryptography," *Scientific Reports*, vol. 16, Art. no. 9582, 2026, doi: 10.1038/s41598-025-28413-8.
- [13] D. C. Lawo, R. Frantz, A. Cano Aguilera, X. Arnal I Clemente, M. P. Podleś, J. L. Imaña, I. Tafur Monroy, and J. J. Vegas Olmos, "Falcon/Kyber and Dilithium/Kyber Network Stack on Nvidia's Data Processing Unit Platform," *IEEE Access*, vol. 12, pp. 38048–38056, 2024, doi: 10.1109/ACCESS.2024.3374629.
- [14] M. A. González de la Torre, I. A. Morales Sandoval, G. T. Freitas de Abreu, and L. Hernández Encinas, "Post-Quantum Wireless-Based Key Encapsulation Mechanism via CRYSTALS-Kyber for Resource-Constrained Devices," *IEEE Access*, vol. 13, pp. 66714–66725, 2025, doi: 10.1109/ACCESS.2025.3560023.
- [15] L. Cruz-Piris, A. Marín-López, M. Álvarez-Campana, M. Sanz, J. I. Moreno, and D. Arroyo, "Measuring the Impact of Post Quantum Cryptography in Industrial IoT Scenarios," *Internet of Things*, vol. 34, Art. no. 101793, 2025, doi: 10.1016/j.iot.2025.101793.
- [16] S. Aghapour, K. Ahmadi, M. Anastasova, M. Mozaffari Kermani, and R. Azarderakhsh, "PUF-Kyber: Design of a PUF-Based Kyber Architecture Benchmarked on Diverse ARM Processors," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 12, pp. 4453–4462, 2024, doi: 10.1109/TCAD.2024.3399669.
- [17] C. Kim, S. Son, and Y. Park, "A Privacy-Preserving Authentication Scheme Using PUF and Biometrics for IoT-Enabled Smart Cities," *Electronics*, vol. 14, no. 10, Art. no. 1953, 2025, doi: 10.3390/electronics14101953.
- [18] C. Rubio García, A. Cano Aguilera, C. Stan, J. J. Vegas Olmos, S. Rommel, and I. Tafur Monroy, "Enhanced Network Security Protocols for the Quantum Era: Combining Classical and Post-Quantum Cryptography, and Quantum Key Distribution," *IEEE Journal on Selected Areas in Communications*, vol. 43, no. 8, pp. 2765–2781, 2025, doi: 10.1109/JSAC.2025.3568011.
- [19] S. Y. Moon, B. H. Jo, A. El Azzaoui, S. K. Singh, and J. H. Park, "Edge-Fog Enhanced Post-Quantum Network Security: Applications, Challenges and Solutions," *Computers, Materials & Continua*, vol. 84, no. 1, pp. 25–55, 2025, doi: 10.32604/cmc.2025.062966.
- [20] D. Aldossary, E. Aldahasi, T. Balharith, and T. Helmy, "A Systematic Literature Review on Load-Balancing Techniques in Fog Computing: Architectures, Strategies, and Emerging Trends," *Computers*, vol. 14, no. 6, Art. no. 217, 2025, doi: 10.3390/computers14060217.
- [21] M. H. Kashani and E. Mahdipour, "Load Balancing Algorithms in Fog Computing," *IEEE Transactions on Services Computing*, vol. 16, no. 2, pp. 1505–1521, 2023, doi: 10.1109/TSC.2022.3174475.
- [22] M. Ala'anzy, R. Zhanuzak, R. Akhmedov, N. Mohamed, and J. Al-Jaroodi, "Dynamic Load Balancing for Enhanced Network Performance in IoT-Enabled Smart Healthcare With Fog Computing," *IEEE Access*, vol. 12, pp. 188957–188975, 2024, doi: 10.1109/ACCESS.2024.3516362.
- [23] S. Fugkeaw, S. Rattagool, P. Jiangthiran, and P. Pholwiset, "FPRESSO: Fast and Privacy-Preserving SSO Authentication With Dynamic Load Balancing for Multi-Cloud-Based Web Applications," *IEEE Access*, vol. 12, pp. 157888–157900, 2024, doi: 10.1109/ACCESS.2024.3485996.
- [24] X. Yu, M. Zhu, M. Zhu, et al., "A Novel Location-Aware Job Scheduling Framework for Optimizing Fog-Cloud IoT Systems: Insights From Dynamic Traffic Management," *Journal of Cloud Computing*, vol. 14, Art. no. 50, 2025, doi: 10.1186/s13677-025-00772-7.

- [25] M. Vijarania, S. Gupta, A. Agrawal, M. O. Adigun, S. A. Ajagbe, and J. B. Awotunde, "Energy Efficient Load-Balancing Mechanism in Integrated IoT-Fog-Cloud Environment," *Electronics*, vol. 12, no. 11, Art. no. 2543, 2023, doi: 10.3390/electronics12112543.
- [26] S. Ijaz, S. G. Ahmad, K. Ayyub, *et al.*, "Energy-Efficient Time and Cost Constraint Scheduling Algorithm Using Improved Multi-Objective Differential Evolution in Fog Computing," *The Journal of Supercomputing*, vol. 81, Art. no. 116, 2025, doi: 10.1007/s11227-024-06550-7.
- [27] J. Wan, B. Chen, S. Wang, M. Xia, D. Li, and C. Liu, "Fog Computing for Energy-Aware Load Balancing and Scheduling in Smart Factory," *IEEE Transactions on Industrial Informatics*, vol. 14, no. 10, pp. 4548–4556, 2018, doi: 10.1109/TII.2018.2818932.
- [28] J. Singh, P. Singh, E. M. Amhoud, and M. Hedabou, "Energy-Efficient and Secure Load Balancing Technique for SDN-Enabled Fog Computing," *Sustainability*, vol. 14, no. 19, Art. no. 12951, 2022, doi: 10.3390/su141912951.
- [29] H. Wang, L. Cai, X. Hao, J. Ren, and Y. Ma, "ETS-TEE: An Energy-Efficient Task Scheduling Strategy in a Mobile Trusted Computing Environment," *Tsinghua Science and Technology*, vol. 28, no. 1, pp. 105–116, 2023, doi: 10.26599/TST.2021.9010088.
- [30] Y. Li, D. Zeng, L. Gu, A. Zhu, Q. Chen, and S. Yu, "PASTO: Enabling Secure and Efficient Task Offloading in TrustZone-Enabled Edge Clouds," *IEEE Transactions on Vehicular Technology*, vol. 72, no. 6, pp. 8234–8238, 2023, doi: 10.1109/TVT.2023.3237204.
- [31] S. Fugkeaw, R. Prasad Gupta and K. Worapaluk, "Secure and Fine-Grained Access Control With Optimized Revocation for Outsourced IoT EHRs With Adaptive Load-Sharing in Fog-Assisted Cloud Environment," in *IEEE Access*, vol. 12, pp. 82753–82768, 2024, doi: 10.1109/ACCESS.2024.3412754.
- [32] Z. Liu *et al.*, "A Privacy-Preserving Outsourcing Computing Scheme Based on Secure Trusted Environment," in *IEEE Transactions on Cloud Computing*, vol. 11, no. 3, pp. 2325–2336, 1 July–Sept. 2023, doi: 10.1109/TCC.2022.3201401.
- [33] Y. Bai, D. He, Z. Yang, M. Luo and C. Peng, "Efficient Module-Lattice-Based Certificateless Online/Offline Signcryption Scheme for Internet of Medical Things," in *IEEE Internet of Things Journal*, vol. 12, no. 14, pp. 27350–27363, 15 July15, 2025, doi: 10.1109/JIOT.2025.3562262.
- [34] W. Zhang, J. Wei and M. K. Khan, "PQC-Enhanced Privacy-Preserving Federated Learning for Edge-Based IoT Ecosystems," in *IEEE Transactions on Consumer Electronics*, vol. 71, no. 4, pp. 11171–11182, Nov. 2025, doi: 10.1109/TCE.2025.3601395.
- [35] A. N. Zaheer, M. Farhan, M. Rehan Naeem and M. M. Alnfai, "Quantum-Resilient Cryptographic Frameworks: Design and Analysis of Post-Quantum Algorithms for Secure and Efficient Edge-Assisted IoT Ecosystems in Consumer Electronics Devices," in *IEEE Transactions on Consumer Electronics*, vol. 71, no. 4, pp. 11258–11268, Nov. 2025, doi: 10.1109/TCE.2025.3619556.
- [36] Z. Man, Z. Yu, J. Yu, C. Gao and X. Meng, "Edge Computing in Internet of Things: Lattice-Based and Split Encryption for Post-Quantum Data Security," in *IEEE Internet of Things Journal*, vol. 12, no. 23, pp. 49327–49339, 1 Dec.1, 2025, doi: 10.1109/JIOT.2025.3591521.
- [37] A. M. Jasim and H. Al-Raweshidy, "An Adaptive SDN-Based Load Balancing Method for Edge/Fog-Based Real-Time Healthcare Systems," in *IEEE Systems Journal*, vol. 18, no. 2, pp. 1139–1150, June 2024, doi: 10.1109/JSYST.2024.3402156.
- [38] R. M. Singh, G. Sikka and L. K. Awasthi, "LBATSM: Load Balancing Aware Task Selection and Migration Approach in Fog Computing Environment," in *IEEE Systems Journal*, vol. 18, no. 2, pp. 796–804, June 2024, doi: 10.1109/JSYST.2024.3403673.
- [39] E. Oustad, "DIST: Distributed Learning-Based Energy-Efficient and Reliable Task Scheduling and Resource Allocation in Fog Computing," in *IEEE Transactions on Services Computing*, vol. 18, no. 3, pp. 1336–1351, May–June 2025, doi: 10.1109/TSC.2025.3568255.
- [40] M. Maashi, "Elevating Survivability in Next-Gen IoT-Fog-Cloud Networks: Scheduling Optimization With the Metaheuristic Mountain Gazelle Algorithm," in *IEEE Transactions on Consumer Electronics*, vol. 70, no. 1, pp. 3802–3809, Feb. 2024, doi: 10.1109/TCE.2024.3371774.
- [41] D. Kang and H. J. Jo, "TB-Logger: Secure Vehicle Data Logging Method Using Trusted Execution Environment and Blockchain," in *IEEE Access*, vol. 11, pp. 23282–23292, 2023, doi: 10.1109/ACCESS.2023.3253626.
- [42] C. Wang, Y. Deng, Z. Ning, K. Leach, J. Li, S. Yan, Z. He, J. Cao, and F. Zhang, "Building a Lightweight Trusted Execution Environment for Arm GPUs," in *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 4, pp. 3801–3816, 2024, doi: 10.1109/TDSC.2023.3334277.
- [43] J. Zhao, H. Zhu, F. Wang, R. Lu, and H. Li, "SXGB: Secure and Efficient Vertical Federated XGBoost via Trusted Execution Environments," in *IEEE Transactions on Dependable and Secure Computing*, vol. 23, no. 2, pp. 2275–2288, Mar. 2026, doi: 10.1109/TDSC.2025.3626379.
- [44] L. Hak and S. Fugkeaw, "SSL-XIoMT: Secure, Scalable, and Lightweight Cross-Domain IoMT Sharing With SSI and ZKP Authentication," *IEEE Open Journal of the Computer Society*, vol. 6, pp. 714–725, 2025, doi: 10.1109/OJCS.2025.3570087.